

From MicroVMs to Userspace MicroKernels: Rethinking Isolation Boundaries in Cloud-Native Systems

Cheng Li

225040518

School of Data Science
The Chinese University of Hong Kong, Shenzhen

April 2026

Abstract

Cloud-native computing relies on densely multiplexed, multi-tenant execution, which sharpens a long-standing tension between isolation strength and runtime overhead. Stronger isolation typically implies heavier startup costs, larger per-instance memory footprints, and longer service paths, while lighter execution environments tend to expose broader attack surfaces or depend on the integrity of a shared host kernel. This report studies three representative responses to that tension: Firecracker, a microVM monitor specialized for serverless workloads; SURE, a unikernel-based serverless framework with a secure high-performance data plane; and LITESHIELD, a userspace microkernel-services architecture for secure containers.

Firecracker specializes the virtualization stack by replacing a general-purpose VMM with a minimal microVM monitor while preserving a conventional Linux guest, so it gains density and startup speed without abandoning compatibility. SURE reorganizes the serverless runtime around a unikernel-based VM, a distributed zero-copy data plane, a library-based sidecar, and Memory Protection Keys (MPK) that together turn secure communication into a first-class fast-path concern. LITESHIELD removes the hypervisor entirely and decomposes much of the guest-kernel functionality into composable userspace microkernel services, contracting the host-facing interface to roughly two dozen syscalls and replacing kernel crossings with shared-memory IPC.

Reading the three papers under a unified six-dimension framework, this report argues that recent work on cloud-native isolation has evolved along three connected axes: the location of the isolation boundary, the width of the application-to-host interface, and the degree to which a runtime specializes its own internal structure. The cross-paper synthesis treats them not as competing answers to one benchmark but as complementary points in a broader design space, and identifies open problems in compatibility, hardware dependence, debuggability, side-channel integration, and unified evaluation that none of the three papers fully resolves alone.

Contents

1	Introduction	1
1.1	Research Questions	2
1.2	Selection of the Three Focal Papers	2
1.3	Contributions and Methodological Posture	3
1.4	Chapter Outline	4
2	Background and Related Work	5
2.1	Why Cloud-Native Isolation Matters	5
2.2	Foundations of Isolation in Modern Systems	5
2.3	Analytical Framework	7
2.4	Representative Stages in the Design Space	8
2.5	Evaluation Focus Across the Three Papers	8
2.6	Hardware Substrates and Their Implications	10
2.7	Caveats and How to Read the Comparisons	10
3	Firecracker	12
3.1	The Lambda Problem and a Design of Subtraction	12
3.2	Anatomy of a MicroVM	13
3.3	Firecracker Inside AWS Lambda	15
3.4	What the Numbers Say	17
3.5	Reading Firecracker as a Design Choice	20
4	SURE	22
4.1	When Cold Start Stops Being the Bottleneck	22
4.2	A Five-Piece Architecture	23
4.3	The Distributed Zero-Copy Data Plane	24

4.4	The Sidecar as a Library	25
4.5	MPK with a Hardened Unikernel TCB	26
4.6	Microbenchmarks and the Online Boutique	28
4.7	What SURE Buys and What It Demands	30
5	LITESHIELD	32
5.1	Re-asking Where the Boundary Should Be	32
5.2	Userspace μ Kernel Services and a Thin Host Interface	33
5.3	Fast IPC and Composable Specialization	34
5.4	Engineering for Compatibility	36
5.5	What the Numbers Show	37
5.6	LITESHIELD as a Reframing, Not a Refinement	39
6	Cross-Paper Synthesis and Open Challenges	41
6.1	Three Axes of Variation	41
6.2	Threat Models in Comparative Form	42
6.3	Recurring Themes Across the Three Papers	43
6.4	Workload Fit and Practical Trade-offs	44
6.5	Open Problems	45
6.6	What These Papers Tell Us About the Field	46
7	Conclusion and Future Work	47

List of Figures

3.1	Security models of Linux containers (a) and KVM-based virtualization (b).	14
3.2	High-level architecture of the AWS Lambda event path.	16
3.3	Architecture of the Lambda worker.	17
3.4	Boot-time CDF for serially-started microVMs across four VMs.	18
3.5	Boot-time CDF for 50 microVMs launched in parallel.	19
3.6	Per-microVM memory overhead vs. configured VM size.	19
4.1	Overall architecture of SURE.	23
4.2	SURE’s intra-node shared-memory data plane.	24
4.3	Library-based sidecar in SURE, with RX and TX hook points.	26
4.4	SURE’s MPK-based call gates between user code and trusted runtime modules.	27
4.5	Online Boutique evaluation, intra-node setting: (a) RPS, (b) response-time CDF, (c) CPU cost per RPS.	29
4.6	Online Boutique evaluation, inter-node setting (panels as in Figure 4.5).	30
5.1	Three representative isolation architectures: VM, unikernel, and container.	32
5.2	Architecture of LITESHIELD.	33
5.3	Fast userspace IPC mechanism in LITESHIELD.	34
5.4	Syscall latency: getpid (simple) vs. read (complex).	37
5.5	Network performance on UDP and TCP across packet sizes.	38
5.6	Filesystem write performance under direct I/O and cached I/O.	38
5.7	YCSB performance on Redis under LITESHIELD.	39

List of Tables

2.1	Representative stages in the evolution of secure cloud-native isolation surveyed in this report.	8
2.2	Evaluation focus across the three focal papers.	9
3.1	Selected quantitative findings reported for Firecracker.	18
6.1	Comparison of the three focal papers under the analytical framework of Chapter 2.	42
6.2	Threat models of the three focal papers, viewed from the cloud provider’s perspective.	43
6.3	Open problems suggested by the three focal papers.	45

Chapter 1

Introduction

Cloud-native computing is now organized around highly dynamic, multi-tenant, and fine-grained execution. Serverless functions, short-lived containers, and loosely coupled microservices all promise operational elasticity, but they also magnify the cost of isolation. The classical trade-off is familiar: heavyweight virtualization buys strong isolation at the cost of startup latency and memory overhead, while shared-kernel containers buy density at the cost of a broad privileged interface and a single-kernel attack surface. The practical question is therefore no longer whether isolation is necessary; it is which isolation architecture best matches cloud-native workloads under modern constraints.

That question is increasingly difficult to answer because the cloud-native workload itself has changed. A modern serverless platform routinely runs millions of short-lived functions, each with small memory budgets and tight latency targets, while a modern microservice deployment can chain together thousands of inter-dependent components whose collective performance depends as much on the data plane as on any single sandbox. In this environment, the isolation mechanism is no longer a static implementation choice; it is a load-bearing piece of system design that interacts with startup behavior, memory density, communication structure, and software compatibility at the same time.

This question motivates the present comparative review. Rather than surveying the entire virtualization literature, this report focuses on three papers that capture distinct design responses to the same broad problem. Firecracker [1] represents a highly specialized microVM architecture for serverless and container workloads, deployed at scale by AWS Lambda and Fargate. SURE [32] represents a unikernel-based serverless framework that combines rapid function startup, zero-copy data-plane communication, library-based sidecars, and intra-runtime memory protection. LITESHIELD [28] represents a userspace isolation architecture that decouples system services into composable microkernel-style components and explicitly removes the hypervisor while keeping a thin interface to the host kernel.

The value of comparing these three papers lies in the fact that they are not merely incremental variants of one another. They redefine where protection should happen, what counts as the trusted computing base, and how much of the traditional kernel or virtualization stack must remain on the critical path. Firecracker asks how far a specialized VMM can be slimmed

down without sacrificing the compatibility that makes Linux a practical guest. SURE asks how a unikernel-based serverless runtime can make function chaining both fast and secure when the dominant cost is no longer cold start but communication. LITESHIELD asks whether secure containers can be reconstructed around userspace service decomposition rather than around another full virtualization layer.

1.1 Research Questions

This review is organized around three research questions. The first question concerns the *location of the isolation boundary* across the three systems. Firecracker places the principal boundary at the hardware-virtualization layer between guest and host, supported by KVM and a minimal VMM. SURE retains a virtualization boundary by running each function as a unikernel-based VM but adds a second, intra-runtime boundary inside the unikernel itself, separating untrusted user code from trusted runtime modules through MPK. LITESHIELD removes the hypervisor and instead enforces isolation by routing application requests through restricted userspace processes and a small set of host syscalls, supplemented by seccomp.

The second question concerns the *interface width and service path* from application to host. Firecracker minimizes the VMM and device model while still exposing a complete Linux guest kernel to workloads, so most application requests still enter a general-purpose kernel. SURE narrows the high-cost paths in serverless data movement through zero-copy intra-node communication, a userspace TCP/IP stack for inter-node traffic, and a library-based sidecar that replaces socket-based service-mesh boundaries. LITESHIELD explicitly targets the user-to-host interface and contracts it to roughly twenty-two syscalls, replacing kernel-mediated calls on the hot path with shared-memory IPC to userspace services.

The third question concerns how much *specialization* is worth its cost. More specialized runtimes promise better startup behavior, higher density, or lower I/O overhead, but they may also depend on stronger assumptions about workload shape, hardware features such as MPK or hardware virtualization, and compatibility requirements. A useful comparative review should evaluate not only headline performance numbers but also the regimes in which each design is plausible to deploy. The three papers studied here make different bets about what is worth specializing, and the value of comparing them lies precisely in surfacing those bets.

1.2 Selection of the Three Focal Papers

The three focal papers were selected because they represent distinct points in the recent evolution of cloud-native isolation. Firecracker is a natural baseline because it has become a widely recognized microVM design for serverless workloads, with publicly documented production

deployment in AWS Lambda and Fargate that has supported millions of workloads and trillions of requests per month [1]. The paper is also methodologically self-contained: it makes its design goals explicit, describes the specialization decisions taken to meet them, and grounds its evaluation in operational experience.

SURE was selected because it extends the conversation beyond startup-focused isolation into the serverless data plane, service mesh, and intra-runtime protection. Where Firecracker accepts a conventional guest-kernel model and tries to shrink the virtualization layer, SURE asks what becomes possible if one builds the runtime around a unikernel and then redesigns communication, sidecar deployment, and memory protection together. Its insistence that secure communication belongs on the fast path makes it a markedly different point in the design space from Firecracker [32].

LITESHIELD was selected because it challenges the assumption that strong isolation for containers must rely on a hypervisor or a full guest kernel. Instead, it proposes a userspace microkernel-services architecture that explicitly removes the hypervisor, decomposes the guest kernel into composable services, and keeps the host-facing interface narrow through seccomp and shared-memory IPC [28]. The paper is especially valuable for comparison because it forces the reader to ask whether the boundary being defended is the right one in the first place.

Taken together, these three papers form a coherent comparative arc. Firecracker emphasizes virtualization specialization, SURE emphasizes secure high-performance serverless communication, and LITESHIELD emphasizes interface minimization and service decomposition. They thus enable a structured comparison across architecture, interface design, compatibility, and deployment fit without being three independent restatements of the same idea.

1.3 Contributions and Methodological Posture

This report makes four contributions. It reorganizes the three focal papers under a common analytical framework rather than presenting them as unrelated summaries, so that their differences become interpretable rather than incommensurable. It compares them along axes that matter for operating-systems design— isolation boundary, trusted computing base, interface width, communication path, compatibility model, and deployment suitability—and shows how the meaning of “lightweight isolation” shifts across microVMs, unikernels, and userspace service architectures. It develops a cross-paper synthesis that identifies open problems in secure cloud-native execution, including evaluation consistency, hardware dependence, debuggability, and operational observability. Underlying these is a more subtle commitment: the three papers are read as exhibits in a design-space discussion rather than as entries in a direct leaderboard-style comparison, because they optimize for different bottlenecks, depend on different hardware substrates, and aim at different deployment regimes—which is also how their original authors position

them.

The methodological posture is interpretive rather than re-implementational. The report does not independently reproduce or benchmark the systems; it analyzes the primary papers, extracts their design logic, compares the assumptions behind their evaluation, and assesses how each paper positions itself relative to the broader tension between security, overhead, and compatibility. The strength of the analysis therefore lies in structured comparison and critical synthesis, and its limitation is equally important: all quantitative claims ultimately depend on the experimental settings reported in the original papers, and direct cross-paper numerical comparison must be treated carefully whenever the workloads, metrics, or hardware are not held constant. Where a number, table, or diagram is informative, the report describes its function and indicates where it appears in the original work; it does not re-execute experiments or invent additional measurements.

1.4 Chapter Outline

Chapter 2 develops the background and related work used throughout the report, including the six-dimension analytical framework that the rest of the report applies. Chapters 3–5 then discuss Firecracker, SURE, and LITESHIELD in turn. The three chapters share the same comparative vocabulary but follow each paper’s own narrative arc rather than a fixed template: Firecracker is read through its operational deployment in AWS Lambda, SURE through its coordinated redesign of runtime and data plane, and LITESHIELD through its argument that the boundary itself should be re-asked. Chapter 6 synthesizes the comparison across the three papers, identifies recurring themes, and discusses open challenges. Chapter 7 concludes with broader implications and possible layered combinations of the three designs.

Chapter 2

Background and Related Work

2.1 Why Cloud-Native Isolation Matters

Multi-tenancy is economically central to modern cloud systems. Containers and serverless functions improve utilization by allowing many workloads to share the same hardware, but the same sharing increases the risk of privilege escalation, information leakage, and noisy-neighbor interference. As a result, isolation is not merely a security property. It is also a performance, operability, and economics problem, and academic surveys of the serverless landscape [16] together with open-source platforms such as OpenLambda [13] have made the resulting trade-offs visible to systems research. An isolation mechanism that is too heavy reduces density and increases startup latency, which directly affects per-request cost. One that is too weak widens the attack surface and complicates production deployment, which directly affects whether the platform can be sold at all.

The three focal papers each emphasize a different aspect of this tension. In Firecracker, the emphasis is on strong isolation with minimal per-workload overhead at large-scale serverless deployment, where memory accounts for roughly 40% of the capital cost of a typical server and where idle slots must remain inexpensive without sacrificing the ability to spin up busy ones [1]. In SURE, the emphasis is that serverless execution is increasingly communication-heavy, so the data plane and service-mesh structure become as important as the sandbox boundary itself [32]. In LITESHIELD, the emphasis is that traditional containers expose too broad an interface to the host kernel and that virtualization can still be too heavyweight or too general-purpose for cloud-native microservices, so the boundary itself should be redesigned [28].

2.2 Foundations of Isolation in Modern Systems

To compare these papers in a principled way, it is useful to recall the four mechanisms on which their designs implicitly or explicitly draw. The first is hardware virtualization, exposed in commodity CPUs through extensions such as Intel VT-x, which lets a hypervisor run guest kernels at reduced privilege while preserving the illusion of a private hardware platform. The dominant

Linux substrate for this mechanism is KVM [18], traditionally paired with the QEMU userspace VMM [6]; Xen [4] represents the alternative type-1 lineage. Firecracker uses KVM but discards QEMU. SURE also uses hardware virtualization, but only as a substrate for unikernel-based VMs. LITESHIELD deliberately does not use it; it relies on process-level isolation enforced by the host kernel and on seccomp-based syscall restriction.

The second is the containerization toolkit built into Linux, including namespaces, cgroups, and seccomp-bpf, embodied in widely used implementations such as Docker. These mechanisms are widely used to multiplex workloads on a shared kernel, but their reliance on the kernel’s syscall interface as the security boundary forces a tension between security and compatibility [1]. Empirical studies have repeatedly shown that even moderate Linux applications require many dozens of syscalls and ioctls to run correctly [38], and that bugs cluster in less-frequently-used kernel paths [26]. Higher-assurance container designs respond either by inserting a hypervisor underneath (e.g., Kata Containers [31]) or by interposing a userspace guest kernel that re-implements much of the Linux interface (e.g., gVisor [40]). LITESHIELD is built on top of namespaces, cgroups, and seccomp but explicitly narrows the syscall interface to roughly twenty-two calls and routes most application requests through userspace services rather than through the host kernel [28]. SURE retains a VM boundary instead.

The third mechanism is the unikernel or library OS, which links application code together with a minimal set of OS libraries into a single address space. The lineage runs from early library-OS designs such as Drawbridge [34], through unikernel platforms such as OSv [19] and Unikraft [24], to systems that position unikernels as a container alternative, such as LightVM [29]. Unikernels can boot in milliseconds, present a small TCB, and offer fast intra-runtime function calls, but they traditionally lack the kernel-userspace separation that protects sensitive runtime components from buggy or malicious user code. SURE adopts a unikernel runtime built on Unikraft and explicitly addresses this protection gap [32].

The fourth is intra-process memory protection, particularly Intel Memory Protection Keys (MPK), which let a process tag pages with one of sixteen protection keys and toggle access privileges on a per-thread basis using the WRPKRU instruction. A growing literature applies MPK to safe binary-rewriting-based isolation [39], with CubicleOS [36] and FlexOS [25] bringing it inside unikernel runtimes. SURE uses MPK to guard shared-memory buffers and protected runtime data structures and to enforce a call-gate abstraction; LITESHIELD does not use MPK directly but relies on process-level separation and shared-memory IPC; Firecracker does not use MPK in the design described in the paper. The presence or absence of MPK in a paper’s threat model is therefore a useful indicator of its specialization strategy.

Beyond these four mechanisms, the broader microkernel literature also shapes the design space. seL4 [20] demonstrated that a small kernel can be formally verified, and Arrakis [33] argued that the OS should become a control plane that delegates I/O to user-level libraries.

LITESHIELD’s userspace μ kernel-services architecture stands in this lineage, although it deliberately works within an unmodified Linux kernel rather than building a new one.

2.3 Analytical Framework

To compare the three systems in a principled way, this report uses six dimensions. First is the *isolation boundary*: where the principal security separation is enforced. Second is the *interface width*: how much of the host or privileged substrate is directly exposed to the workload. Third is the *service-path structure*: whether application requests traverse syscalls, VM exits, userspace IPC, sidecars, shared memory, or specialized protocol stacks before they reach a result. Fourth is the *compatibility model*: whether unmodified Linux binaries can run, whether guest-kernel assumptions are preserved, and how much porting effort is required. Fifth is the *specialization strategy*: whether the system minimizes general components, decomposes services, or tailors itself to a narrow workload class. Sixth is the *deployment target*: whether the architecture is best suited for general secure containers, serverless functions, or communication-heavy microservices.

Compactly, an execution environment can be modeled as a tuple $E = (B, I, P, C, Z, D)$ corresponding to the six dimensions enumerated above: B for the isolation boundary, I for interface width, P for service path, C for compatibility, Z for specialization strategy, and D for deployment target. A choice of E implies a particular trade-off among the qualities the dimensions induce—chiefly isolation strength, runtime overhead, and compatibility. Firecracker prioritizes isolation strength and broad Linux compatibility under serverless scale, while accepting that the hot path still crosses a guest kernel and a virtualization boundary. SURE prioritizes low-latency communication and secure function chaining, at the cost of a more specialized unikernel runtime that depends on MPK. LITESHIELD prioritizes a thin host interface and the absence of a hypervisor, while accepting that some non-delegable syscalls must be trapped through ptrace and that the userspace service layer carries its own engineering burden. The framework is not a quantitative theory; its purpose is to make explicit that the three papers are not pursuing the same optimization with different parameters but operating in different regions of the same loose objective. This is also why all three can plausibly claim to be both secure and lightweight: Firecracker is lightweight because it removes most of what a general-purpose VMM contains, SURE because it eliminates kernel-mediated socket communication on the hot path and keeps the sidecar inside the function’s address space, LITESHIELD because it eliminates the hypervisor entirely and answers most syscalls in userspace. Each statement is true on its own terms, and the framework helps the reader avoid conflating them.

2.4 Representative Stages in the Design Space

Table 2.1 summarizes the broad design progression represented by the three focal papers.

Table 2.1: Representative stages in the evolution of secure cloud-native isolation surveyed in this report.

Stage	System	Main design move	Limitation addressed
MicroVM specialization	Firecracker	Replace a general-purpose VMM with a minimal microVM monitor while preserving Linux guest compatibility.	Cut virtualization overhead and startup time without weakening isolation.
Unikernel data-plane specialization	SURE	Add zero-copy intra- and inter-node communication, a library-based sidecar, and MPK-protected runtime modules.	Cut communication overhead and protect runtime components from user code.
Userspace service decomposition	LITESHIELD	Move guest-kernel functionality into composable userspace μ kernel services and remove the hypervisor.	Narrow the host interface and avoid hypervisor complexity.

2.5 Evaluation Focus Across the Three Papers

The three papers also differ in what they treat as decisive evidence. Firecracker emphasizes density, startup time, memory overhead, and real production deployment. The paper reports memory overhead of roughly 3MB per microVM compared with about 131MB for QEMU, boot to application code in around 125ms, sustained creation of up to 150 microVMs per second per host, and overcommit ratios above 20 $\times$ in testing and around 10 $\times$ in production [1]. SURE emphasizes throughput, data-plane efficiency, sidecar overhead reduction, and the cost of intra-runtime memory protection. The paper reports up to 79 $\times$ throughput improvement over Knative on a representative microservice workload, more than 100 $\times$ CPU-cycle savings and roughly 16 $\times$ throughput improvement from its library-based sidecar relative to an individual userspace sidecar, and a roughly 4 $\times$ faster boot than Docker containers [32]. LITESHIELD emphasizes interface reduction, implementation complexity, and performance parity with tradi-

tional containers. The paper reports that the system narrows the host-facing syscall interface to twenty-two calls, supports about 170 syscalls in userspace, and delivers performance comparable to traditional containers (and in some workloads, such as Redis under YCSB, even slightly above native) [28].

Table 2.2 summarizes the evaluation logic of the three papers in compact form.

Table 2.2: Evaluation focus across the three focal papers.

Paper	Primary metrics	Benchmark style	What the evaluation is really testing
Firecracker	Boot time, per-VM memory overhead, IO throughput, density, oversubscription	Microbenchmarks plus production experience in AWS Lambda and Fargate	Whether a specialized microVM monitor can match container-level density and startup at VM-level isolation
SURE	Latency, throughput (RPS), CPU cost per request, sidecar overhead, MPK overhead, boot time	Microbenchmarks plus the Online Boutique microservice application against Knative, SPRIGHT, NightCore, and serverful baselines	Whether a unikernel-centered runtime with zero-copy communication and MPK-protected TCB can dominate kernel-mediated serverless platforms on communication-heavy workloads
LITESHIELD	Syscall latency, network throughput, file I/O, real-application performance, interface size	Microbenchmarks plus Redis under YCSB against Docker, KVM, Firecracker, and gVisor	Whether userspace microkernel services with shared-memory IPC can approximate VM-class isolation at container-class performance

These differences matter because they shape what each paper means by “efficient” or “lightweight,” and the careful reader should resist treating any one of the three as the universal answer.

2.6 Hardware Substrates and Their Implications

The three systems also differ in which hardware features they assume. Firecracker depends on hardware virtualization extensions exposed through KVM but does not require Intel MPK or any specific accelerator. SURE depends on both hardware virtualization (because each function is a VM) and Intel MPK (for memory-level protection of the shared address space and TCB), and its data plane is co-designed with DPDK polling for high-throughput packet handling. LITESHIELD does not require hardware virtualization at all, since its isolation is enforced by Linux’s process boundary, seccomp, and ptrace; instead, its performance argument depends on multi-core systems with last-level caches large enough to make cache-to-cache shared-memory IPC competitive with syscalls.

This contrast is methodologically important. A paper’s claims travel only as far as its hardware substrate. SURE’s protection mechanism depends on x86 MPK support, so its portability is constrained by the availability and behavior of that hardware feature on the target platform. LITESHIELD’s polling thread design assumes that dedicating cores to userspace services is acceptable. Firecracker’s density story assumes commodity virtualization-capable servers but is otherwise relatively portable.

2.7 Caveats and How to Read the Comparisons

The framework above is a tool for structured synthesis, not a leaderboard. The report does not attempt a full history of containers, hypervisors, unikernels, or microkernels, nor does it survey adjacent work on language-runtime isolation or formally verified microkernels except where that work clarifies a focal paper’s argument. All quantitative claims rely on the figures, tables, and numbers reported in the original papers; figures reproduced here are imported from those sources for ease of reference rather than re-measured.

Cross-paper number-to-number comparison must therefore be treated carefully for at least three reasons. The target abstractions differ—Firecracker is a microVM monitor, SURE is a serverless framework, LITESHIELD is a secure-container architecture—so asking “which is fastest?” without naming the workload and the hardware is close to meaningless. The hot paths differ: Firecracker centers on booting and hosting microVMs, SURE on inter-function communication and service-mesh overhead, LITESHIELD on syscall delegation and userspace IPC. And the compatibility assumptions differ: Firecracker preserves a conventional guest-kernel model, SURE asks applications to be rebuilt against unikernel libraries, and LITESHIELD inserts an interception and arbitration layer that must remain compatible with unmodified Linux binaries. None of this invalidates comparison; it simply requires the analysis to focus on design dimensions rather than headline numbers alone. Where the three papers do allow a clean

comparison—for example on memory footprint, startup time, or interface width—the comparison is informative precisely because the three systems made different design choices that should produce different numbers. The remainder of the report uses the framework of this chapter to make those design choices explicit, so that the next three chapters can be read as differing answers to the same family of questions rather than as competing entries in one ranking.

Chapter 3

Firecracker

3.1 The Lambda Problem and a Design of Subtraction

Firecracker starts from a practical cloud-provider problem rather than from an abstract virtualization question. Serverless and container workloads require strong isolation, but conventional virtualization stacks are often considered too heavy for the densities and switching rates these workloads demand. Linux containers are lightweight but rely on a shared host kernel and therefore force uncomfortable trade-offs between security and compatibility [1]. Empirical measurements of Linux API usage confirm that even moderate Ubuntu installations need more than two hundred unique syscalls and more than fifty unique ioctls to run common applications without problems [38], so blanket syscall restriction is not a free lever. The motivating setting is especially clear in public clouds, where many tenants share hardware, workloads must be packed densely while remaining protected against privilege escalation, information disclosure, and side channels, and idle workloads must remain cheap to keep on standby. AWS Lambda, the paper’s primary deployment context, allows individual functions to use as little as 128MB of RAM, which means that a single host with up to 1TB of RAM may need to hold thousands of concurrent functions even before considering soft allocation. In that regime, every megabyte of per-VM overhead matters.

Faced with this problem, Firecracker articulates six design goals—isolation, overhead and density, performance, compatibility, fast switching, and soft allocation [1]—and answers them with a design of subtraction. Density and overhead push toward lighter abstractions, while isolation and compatibility push back toward full virtualization; the paper resolves this by removing features common in general-purpose VMMs but inessential for the intended deployment model. Firecracker offers no BIOS or general firmware boot path, supports only a narrow class of directly-loaded Linux kernels under a constrained configuration, does not emulate legacy devices or PCI, and does not support VM live migration. The codebase is roughly 50,000 lines of Rust, compared to QEMU’s more than 1.4 million lines—evidence the paper marshals to argue that broad scope is in tension with both security and maintainability. The process-per-VM model also simplifies isolation reasoning and operational control, because standard Linux tools such as `ps`, `top`, and `kill` continue to work on microVM hosts.

The key philosophical move is to keep KVM [18] and replace only the VMM and device model. KVM is a battle-tested in-kernel virtualization layer that benefits from broad operational experience and from the wider Linux ecosystem; the QEMU userspace VMM [6] has historically been the heavyweight component, with most of the device model and a long list of features unneeded in a serverless setting. By subtracting from QEMU rather than from KVM, Firecracker preserves the parts of the stack that already work well and rewrites the parts most in tension with cloud-native goals. The implicit security claim that the “popular paths” of the host kernel are the trustworthy ones is reinforced by independent studies of where Linux kernel bugs cluster [26].

The choice of Rust as the implementation language compounds with the codebase-size argument. A long line of QEMU CVEs over the past decade have been memory-safety bugs in C code—use-after-free, out-of-bounds writes, and missing length checks in device emulation paths—that translate directly into VMM-escape primitives in a multi-tenant cloud. By writing Firecracker in Rust, the team eliminates an entire class of bugs from the userspace component that talks directly to KVM, which is the piece of code best positioned to turn a memory-safety bug into a sandbox escape. The argument is therefore not just “50K lines instead of 1.4M”; it is “50K lines of memory-safe code instead of 1.4M lines of unsafe C,” which compresses the auditable surface by orders of magnitude across two independent dimensions.

3.2 Anatomy of a MicroVM

The microVM abstraction is the technical artifact that this design philosophy yields. With a minimal Linux guest kernel configuration, the paper reports less than 5MB memory overhead per container, boot to application code in less than 125ms, and support for the creation of up to 150 microVMs per second per host [1]. These figures matter because they address the two classic objections to VM-based isolation in serverless systems: boot delay and per-instance overhead. Crucially, Firecracker runs an unmodified (though minimal) Linux kernel and userland, preserving a familiar guest environment rather than requiring applications to be ported into a radically different abstraction such as a unikernel [19, 24, 29] or a managed-language VM. This compatibility posture is one of its strongest practical advantages relative to the more specialized runtimes that SURE and LITESHIELD later propose.

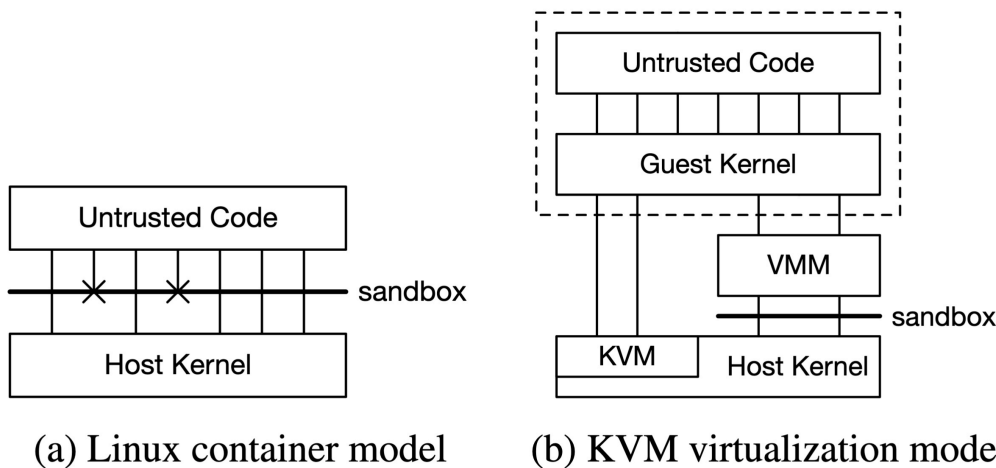


Figure 3.1: Security models of Linux containers (a) and KVM-based virtualization (b).

The device model surfaces this same minimalism: network and block devices, serial ports, and partial i8042 (PS/2 keyboard controller) support [1]. Network and block devices ride on virtio, with the entire block implementation taking only around 1,400 lines of Rust including MMIO and data structures. Virtio is itself a deliberate choice: it is an open paravirtualization standard with a decade of optimization in the Linux kernel, so the guest already has well-tested drivers and Firecracker only has to supply the hypervisor side. Filesystem passthrough is deliberately omitted, because filesystems are large complex codebases and exposing block IO to the guest protects a substantial part of the host kernel surface. Configuration happens through a REST API over a Unix domain socket, which enables fine-grained life-cycle control such as pre-configuring a microVM and only starting it when needed. The choice of REST is operationally motivated rather than aesthetic: clients exist in nearly every language ecosystem, OpenAPI provides machine-readable specifications, and the design slots cleanly into existing orchestration systems such as containerd and Kubernetes [23] rather than competing with them. The same API also exposes cpuid masking, which lets the host hide CPU features from the microVM. This matters less for an isolated function than for fleet management: by presenting a homogeneous CPU profile across heterogeneous hardware, the platform can place slots on any worker without surprising guest software that has feature-detected against a specific CPUID. A telling secondary detail is rate limiting: built-in token buckets for block and network devices sit close to device emulation, where they can directly bound the VMM and host-kernel CPU time a guest can consume, instead of trusting cgroups inside the guest to police themselves. This is the design philosophy in miniature—add a feature to Firecracker only when there is a compelling reason and otherwise rely on the host OS.

Around this minimal core, Firecracker stacks two reinforcing security layers. The first is the process-per-VM model: each microVM is just another Linux process, inheriting standard process isolation and operational tooling. The second is the jailer, which wraps Firecracker in an additional sandbox before it boots the guest. The jailer places Firecracker into a chroot,

isolates it in pid and network namespaces, drops privileges, and applies a restrictive seccomp-bpf profile that whitelists 24 syscalls (with argument filtering) and 30 ioctls, of which 22 are required by KVM’s ioctl-based API [1]. The 24 syscalls themselves are a useful proxy for what a Firecracker process actually does: process and thread management, memory mapping, signal handling, narrow file I/O on tightly limited paths, and the KVM ioctl interface for VMM operations. The chroot contains only the Firecracker binary, `/dev/net/tun`, cgroup control files, and the storage image of the microVM it manages—enough to operate, but stripped of any pathway by which a compromised VMM could read other tenants’ data. The result is a small host-facing syscall interface, even though the workload itself still runs against a full Linux guest kernel inside the microVM. This last point is worth holding onto for the cross-paper comparison: SURE later keeps the host surface narrow as well but adds a second, intra-VM boundary using MPK, while LITESHIELD removes the hypervisor entirely and instead contracts the user-to-host syscall surface to about twenty-two calls. Each paper therefore makes its security argument on a different layer of the stack.

3.3 Firecracker Inside AWS Lambda

Firecracker’s deployment story in AWS Lambda is one of the paper’s distinctive contributions. The Lambda worker runs hundreds or thousands of microVMs per server, each providing a single slot for a single function and customer. Slots transition between init, idle, and busy states, and Lambda relies on soft allocation and multi-tenancy to keep idle slots cheap and busy slots well-served [1]. The MicroManager process per worker is responsible for managing the Firecracker processes on that worker, and a Worker Manager and Placement service coordinate the routing and placement of slots across the fleet. Soft allocation is not just an optimization but a necessary condition for the platform’s economics: idle slots consume only memory (about 40% of a server’s capital cost), so packing many idle slots onto a host is cheap, but a busy slot temporarily needs CPU, network, and cache bandwidth as well. Multi-tenancy reduces this risk approximately as $\sqrt{N}$ for N uncorrelated workloads, which is why Lambda places *unrelated* customers on the same host rather than packing each customer’s workloads together.

Each Lambda slot exists for at most twelve hours before it is recycled, and this recycling doubles as a security hygiene mechanism: any compromise that requires accumulating state—memory disclosure attacks, time-based side channels, slow-leak primitives—runs into a hard upper bound on dwell time. The same mechanism is what made the Firecracker rollout possible at all. AWS migrated Lambda customers between the legacy isolation stack and Firecracker by changing the recycling logic so that newly created slots ran on Firecracker-backed workers; customers experienced no downtime, since their requests landed on whichever stack happened to be active at the time of slot creation. The migration began with internal AWS workloads, where the team had access to detailed metrics and could detect anomalies that an external customer might

not surface, and only afterward extended to external customers behind per-workload monitoring and an automatic rollback path. The paper notes one such rollback prompted by DNS-related performance regressions, traced to a misconfigured DNS-cache setting in the new microVM image. The lesson, easy to miss in an architecture-first reading, is that the cost of migration in a service like Lambda is dominated by operational uncertainty rather than by code differences.

The paper notes another practical detail often overlooked in academic work: a small pool of pre-booted microVMs is maintained to hide cold-start latency from customers. Even though Firecracker boots in around 125ms, that is not always fast enough for the scale-up path of Lambda, which is sometimes blocking user requests. By Little's law, the required mean pool size is the product of the creation rate and creation latency, so faster boots directly reduce the size of the pool the platform must keep warm. At 125ms creation time, one pooled microVM is required for every eight creations per second; halving the boot time would halve the pool. This detail is a useful reminder that startup time is not just a per-instance metric; it shapes the entire warm-pool strategy of a serverless platform.

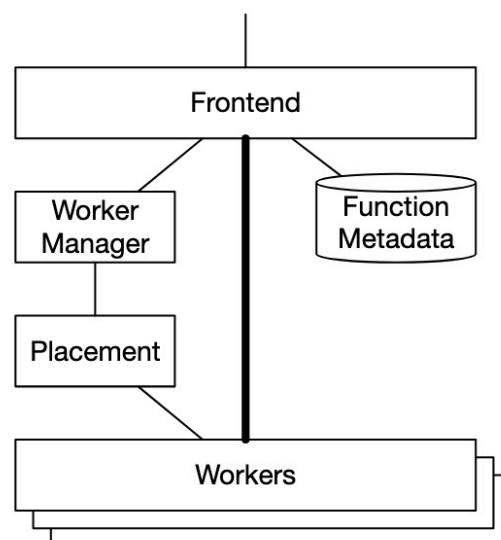


Figure 3.2: High-level architecture of the AWS Lambda event path.

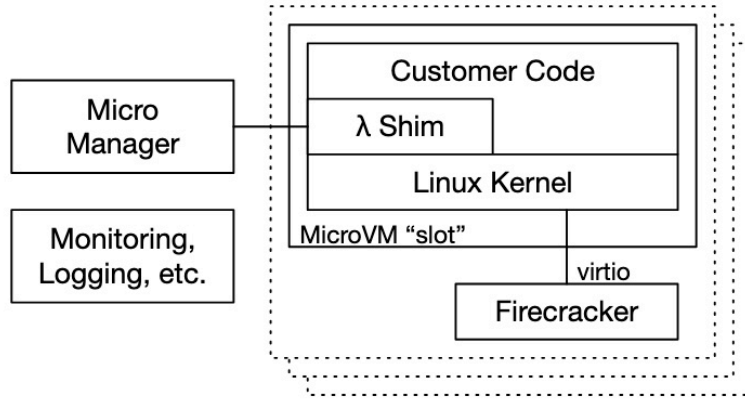


Figure 3.3: Architecture of the Lambda worker.

3.4 What the Numbers Say

The evaluation in the Firecracker paper is organized around the goals the system sets for itself: isolation, overhead and density, performance, compatibility, fast switching, and soft allocation. Three tables and several figures are particularly informative. The measurements are taken on EC2 `m5d.metal` instances with two Intel Xeon Platinum 8175M processors and 384GB of RAM running Ubuntu 18.04.2 with kernel 4.15, and the comparators are QEMU and Intel’s Cloud Hypervisor—two userspace VMMs of similar shape [1].

Table 3.1 summarizes the central quantitative findings reported in the paper. Boot times are measured wall-clock from process fork to the guest’s init process executing, with VMs configured with one vCPU and 256MB of memory and using a minimal init that simply writes to an IO port. Memory overhead is measured as the difference between the memory used by the VMM process and the configured microVM size, summing non-shared memory segments reported by `pmap`. The paper additionally reports IO performance for both block and network devices, and acknowledges that the virtio-based path with serial IO handling does not match the bare-metal numbers possible with PCI pass-through.

Table 3.1: Selected quantitative findings reported for Firecracker.

Metric	Reported value	Comparative context
Memory overhead per microVM	$\approx 3\text{MB}$	QEMU $\approx 131\text{MB}$; Cloud Hypervisor $\approx 13\text{MB}$
Boot time to application code	$\approx 125\text{ms}$	Faster than QEMU on the same setup; comparable to or below Cloud Hypervisor
Sustained microVM creation rate	up to 150 microVMs / second / host	Bounds warm-pool size for Lambda
Codebase size	$\approx 50,000$ lines of Rust	QEMU > 1.4 million lines
Host-facing syscall surface (jailer)	24 syscalls plus 30 ioctls	Compare with > 200 syscalls for typical Linux applications
Soft allocation tested / production	$> 20\times$ tested, up to $10\times$ production	Evidence that overcommit is feasible at cloud scale

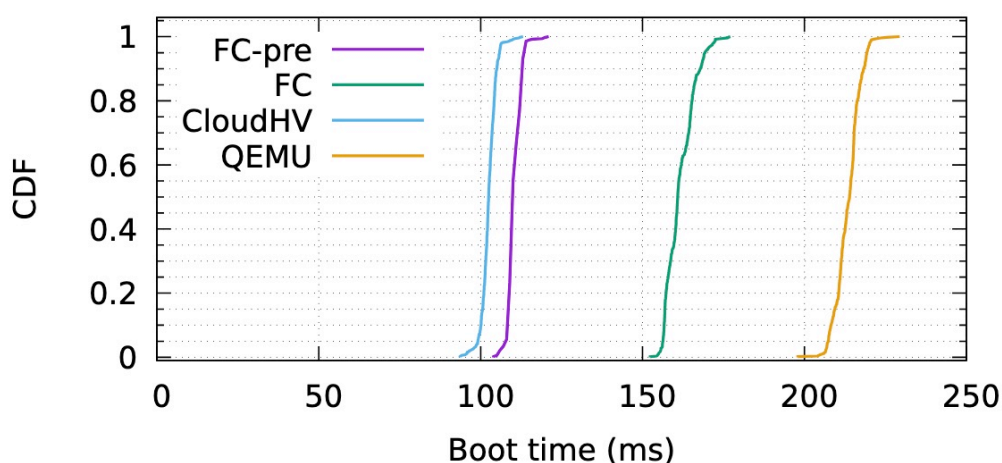


Figure 3.4: Boot-time CDF for serially-started microVMs across four VMMs.

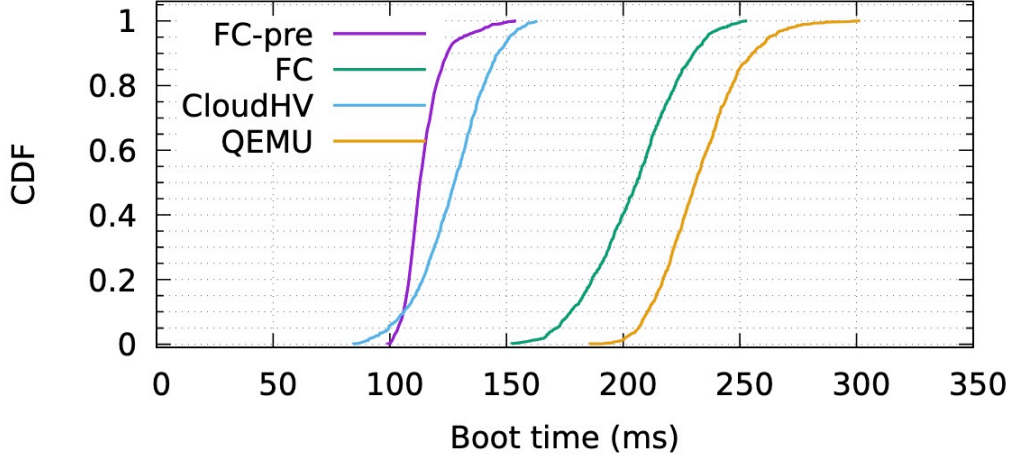


Figure 3.5: Boot-time CDF for 50 microVMs launched in parallel.

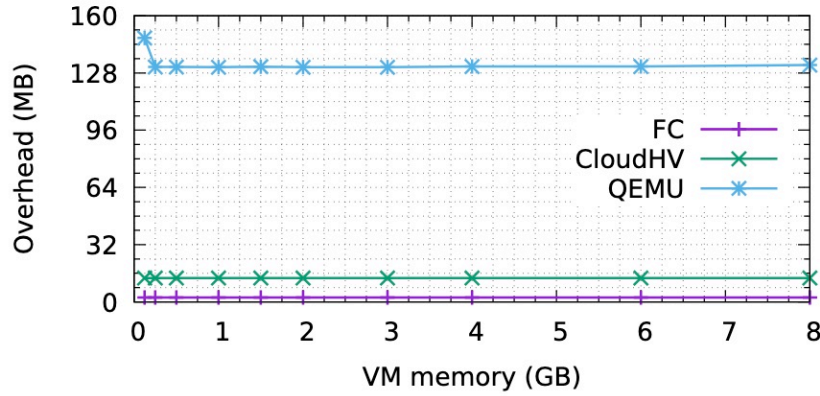


Figure 3.6: Per-microVM memory overhead vs. configured VM size.

The serial and parallel boot CDFs together carry a subtler message about how Firecracker scales. Pre-configured Firecracker boots roughly 50ms faster than the end-to-end variant in serial mode, and the gap remains essentially constant when 50 microVMs are launched in parallel: the 99th percentile shifts only from about 146ms to 153ms. In other words, contention from concurrent boots does not appreciably degrade the tail. This matters because warm-pool refresh in Lambda happens in bursts whenever a function suddenly scales up, and a VMM whose boot tail blew up under concurrency would force the platform to hold larger standby pools than Little’s law alone requires. The memory-overhead numbers compound at scale in the same direction: at the densities Lambda targets, a 1TB host running 8000 of the smallest 128MB functions sets aside 1024GB for guest memory, and QEMU’s 131MB-per-VM overhead would consume an additional roughly 1024GB—doubling the per-host memory budget for no application benefit. Firecracker’s ≈ 3 MB overhead becomes negligible at the same scale, and Cloud Hypervisor’s ≈ 13 MB sits about $4\times$ above Firecracker, enough that the gap is itself a competitive lever among microVM monitors. The IO performance trade-off is the price the design pays for these gains:

virtio with serialized request handling cannot match PCI pass-through, but it preserves the host kernel surface that an isolation-first design needs to defend.

The deployment-grounded character of the evaluation gives Firecracker a distinctive position among systems papers. Its claims are not only architectural but also operational: Firecracker has been running in AWS Lambda since 2018 and supports millions of workloads and trillions of requests per month, with seamless migration of customers between the legacy stack and Firecracker through ordinary slot recycling [1].

3.5 Reading Firecracker as a Design Choice

Read together, the design and the numbers make a coherent argument with four pillars. The first is the compromise between compatibility and isolation: by retaining a Linux guest, Firecracker avoids the application-porting burden of more specialized runtimes while still claiming VM-class isolation. The second is the discipline of subtraction: a minimal device model and reduced feature scope make the design easier to reason about than a general-purpose VMM, both for security and for engineering. The third is the alignment between metrics and operational concerns: boot time, per-instance memory, density, and switching rate are not abstract microbenchmark targets but the same numbers that determine per-request economics in Lambda. The fourth, and the most distinctive, is the production deployment story—a paper that explains how a system was migrated into a real Lambda fleet, including the rollback strategy and the use of internal customers as early adopters, is rare in the systems literature and makes the design’s claims much harder to dismiss. A subtler point reinforces all four: Firecracker chooses where to be conservative. It does not try to replace KVM, Kubernetes, Docker, or the Linux scheduler. It targets only the VMM and device model. Independent comparative studies of microVM and userspace-kernel approaches [3] have since reinforced the view that this focus is itself a contribution.

The same choices, however, mark the boundary of Firecracker’s argument. It remains a virtualization-based design, so it does not eliminate the guest kernel or the costs of a VM abstraction. Its IO performance, while adequate for Lambda and Fargate, is not on par with PCI pass-through, and the paper itself acknowledges room for improvement in storage and network paths [1]. The minimal feature envelope—no BIOS, no arbitrary kernels, no legacy emulation, no migration—is what makes Firecracker work for serverless, but it is a specialization choice rather than a universal replacement for general virtualization. Workloads that depend on capabilities Firecracker has chosen not to provide—live migration for long-running services, PCI pass-through for hardware accelerators, GPU-attached compute, or general-purpose workstation guests—fall outside its envelope by design. This is not an accidental limitation; it follows from the same subtraction discipline that makes the rest of the system work. A platform that wants

both Lambda-class density and HPC-class accelerator access has to either stack Firecracker beneath a more capable layer or pick a different VMM altogether. More importantly, Firecracker leaves untouched two questions that the other papers in this report take seriously. It does not narrow the syscall interface inside the guest, because the guest still runs a full Linux kernel—a surface LITESHIELD will later argue is the right place to push back on. And it does not provide intra-runtime memory protection between user code and trusted runtime modules—an absence that SURE will treat as the central problem once communication-heavy workloads bring sidecars and shared memory into the same address space. Microarchitectural side-channel attacks such as Meltdown [27] and Spectre [22], along with related transient-execution attacks, cut across virtualization-based isolation more broadly, and a comprehensive defense across these classes [9] is necessarily layered across hardware, microcode, kernel, and VMM rather than provided by Firecracker alone.

The right way to position Firecracker, then, is as an exercise in virtualization minimization under real cloud constraints rather than as a redefinition of the isolation boundary. It accepts the boundary the hypervisor industry has provided and asks what can be removed above it. This is precisely the assumption the next two papers question: SURE asks what becomes possible if the runtime above that boundary is reorganized as a unikernel with secure communication and intra-runtime protection, and LITESHIELD asks whether that boundary should even exist for cloud-native containers in the first place. In that sense, Firecracker is the load-bearing baseline of the comparative arc developed in this report—quantitatively impressive within its own design regime and operationally validated by AWS Lambda and Fargate, but also a starting point that leaves the questions of communication, intra-runtime protection, and host-interface width still open.

Chapter 4

SURE

4.1 When Cold Start Stops Being the Bottleneck

SURE addresses a different bottleneck from Firecracker. Its starting point is that serverless platforms increasingly execute loosely coupled microservices organized as function chains, so performance is no longer determined only by sandbox startup. It is also shaped by inter-function networking, sidecar overheads, and secure data movement across nodes [32]. Studies of host network stack overheads [8] and of the microsecond-scale latency targets that increasingly dominate cloud workloads [5] reinforce the argument that the data plane is now a first-order systems concern: kernel-based networking, duplicate protocol processing, sidecar boundary crossings, and serialization can together account for more than half of a function’s CPU budget when the function itself is small. Production traces from large-scale microservice deployments report dependency graphs containing thousands of interdependent services per request [32], so each additional microsecond on the inter-function path multiplies through the chain. The paper argues that containerized serverless runtimes suffer from insufficient isolation and excessive startup time, while naive unikernel-based designs still inherit slow communication paths or weak intra-runtime protection. Existing serverless platforms, including the open-source Knative [21], the eBPF-accelerated SPRIGHT [35], and the engine-mediated NightCore [15], each address a slice of this problem but leave gaps that SURE seeks to close. The change in framing—from “how to sandbox a function” to “how to build an efficient and secure serverless runtime for communicating functions”—is what makes SURE complementary to, rather than competitive with, Firecracker. Firecracker assumes the bottleneck is the per-VM cost; SURE assumes it is the per-message cost in chains of small functions.

From this framing the paper extracts four design implications that organize the rest of its contribution. First, although unikernel-based VMs already offer fast startup, they require enhanced intra-unikernel isolation to protect TCB components from user code. Second, zero-copy networking should extend across nodes so that high performance is not limited to colocated functions. Third, secure shared-memory processing requires fine-grained access control at the level of individual message buffers, not only at the level of group-based memory pools. Fourth, an ideal sidecar should provide L7 visibility and not be constrained by eBPF bytecode limits,

while still preserving the event-driven, low-overhead character that has made eBPF-based service meshes attractive [32]. Each implication corresponds to a concrete design move described in the next sections.

4.2 A Five-Piece Architecture

SURE deploys each serverless function as a unikernel-based VM, called a SURE VM, to combine rapid function startup with per-function VM-style isolation [32]. The architecture does not stop there. It introduces a distributed zero-copy data plane that combines shared memory for intra-node traffic with a userspace zero-copy protocol stack, Z-stack, for inter-node traffic [32]. A per-node SURE gateway coordinates inter-node communication and consolidates protocol processing for colocated functions [32].

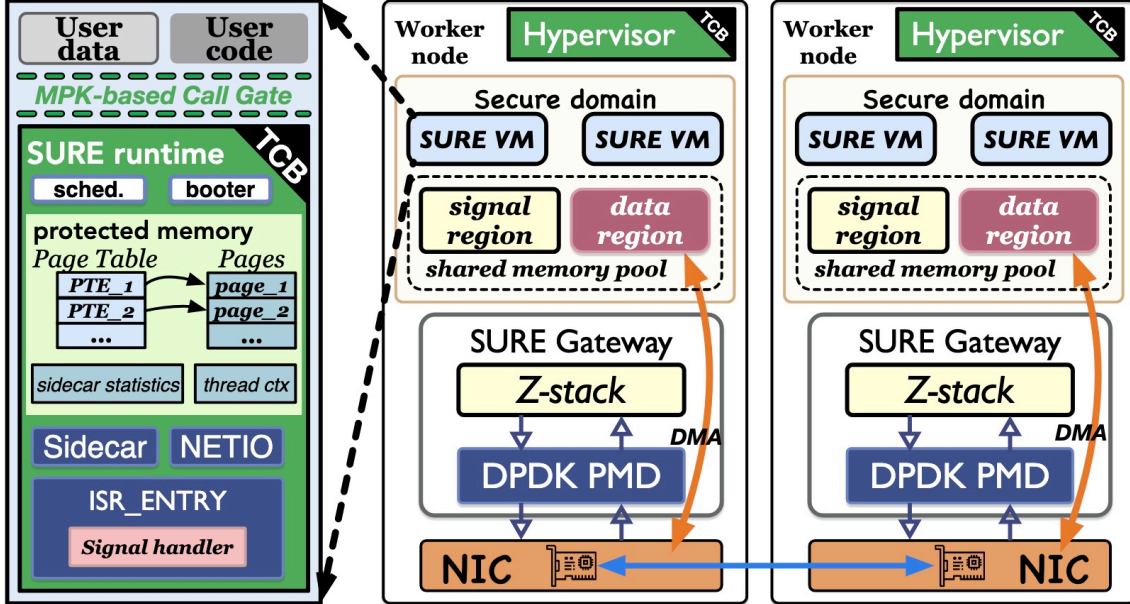


Figure 4.1: Overall architecture of SURE.

The paper distinguishes between mutually trusted SURE VMs (typically belonging to the same user) that are placed in the same security domain, and untrusted VMs from different users that must be separated. Each security domain owns a private shared-memory pool, while cross-domain communication on the same node is mediated by a copy at the SURE gateway, and inter-node communication crosses through Z-stack at both endpoints.

4.3 The Distributed Zero-Copy Data Plane

The intra-node data plane in SURE is built around shared memory. Each VM has access to a signal region, which contains connection contexts, descriptor rings, and a signal ring used to wake up peer VMs through inter-VM interrupts handled by an interrupt service routine. Actual message payloads live in pre-allocated buffers in a separate data region, and ownership is transferred between VMs by exchanging descriptors rather than copying data [32]. Reliable in-order delivery and back-pressure are realized through a FIFO descriptor ring and through `waiting_send/waiting_rcv` pointers that allow a sender to block when a receiver is full and a receiver to block when no descriptors are available.

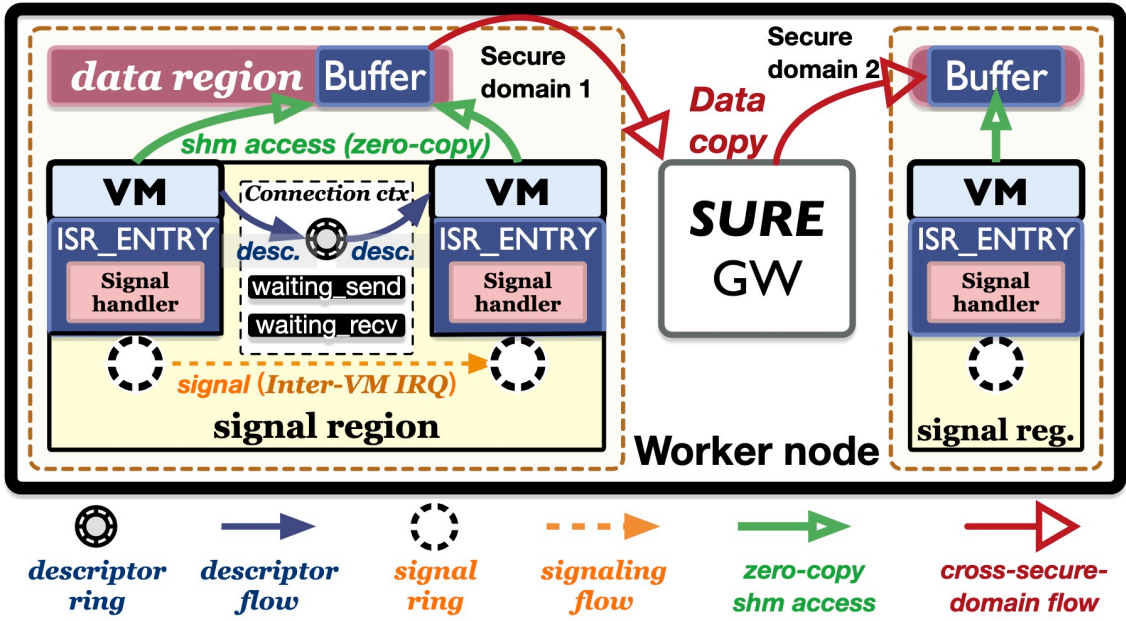


Figure 4.2: SURE’s intra-node shared-memory data plane.

For inter-node traffic, SURE introduces Z-stack, a zero-copy userspace TCP/IP stack built on top of the DPDK F-stack [12]. Z-stack adds two zero-copy APIs (`z_rcv` and `z_send`) that exchange pointers between the application layer and the protocol layer, eliminating the copies that ordinary POSIX socket interfaces incur when moving data between application buffers and socket buffers [32]. The result is that the SURE gateway can DMA packets directly between the NIC and the shared memory pool through the DPDK Poll Mode Driver, never going through a kernel protocol stack. The broader literature on userspace data planes, exemplified by Snap [30], establishes that bypassing the kernel TCP/IP stack can yield significant gains, but most such efforts lack either the lightweight isolation or the manageability that a serverless platform requires. Routing tables in SURE are stored in the signal region of each security domain and are read-only to SURE VMs but writable by the hypervisor when new connections are established.

The combined design achieves end-to-end zero copy: intra-node communication moves de-

scriptors and never copies data, and inter-node communication is processed entirely in userspace and DMAed in and out of shared memory. This is a meaningful step beyond previous shared-memory serverless designs that either keep cross-node traffic on the kernel stack or do not perform consolidated protocol processing.

The data plane also has to handle two failure modes that pure shared-memory schemes typically ignore. The first is back-pressure: a fast sender must not be allowed to overrun a slow receiver, but blocking the sender on every full ring would serialize the chain. SURE's connection context exposes per-connection `waiting_send` and `waiting_recv` pointers that let one side register interest and go to sleep while the other side wakes it up exactly when capacity becomes available, so the cost of back-pressure is paid only when the queue actually fills. The descriptor ring is FIFO, which preserves message ordering without a separate sequence-number protocol. The second failure mode is the SURE gateway itself: because all cross-domain and cross-node traffic flows through it, a crashed gateway would stall every chain on the node. SURE addresses this by running multiple gateway replicas, each handling a subset of the NIC's receive queues through Receive Side Scaling. When one replica fails, the platform reprograms the RSS indirection table to redirect that queue's traffic to a healthy replica and restart the failed one, so the failure window is bounded by detection latency rather than by re-establishment of all in-flight connections.

4.4 The Sidecar as a Library

Service-mesh sidecars are a notorious source of overhead in microservice deployments. Container-based sidecars in production deployments such as Istio [14] typically attach as separate processes that communicate with the user function through TCP/IP sockets, sometimes accelerated by Unix domain sockets; even with the loopback fast path, every request crosses kernel/userspace boundaries at least four times—function out, sidecar in, sidecar out, peer in—and every crossing carries the usual costs of context switching, cache pollution, and per-syscall accounting. eBPF-based sidecars avoid the boundary crossing because their bytecode runs in the kernel itself, but the same in-kernel execution forces them to satisfy the eBPF verifier's safety constraints: bounded loops, no dynamic memory allocation, limited stack depth, and restricted access to packet contents. These constraints make L7 functionality (HTTP/gRPC parsing, header rewriting, payload-aware policies) extremely difficult to express in eBPF without escape hatches into userspace—the very escape hatches that re-introduce the overhead the design was meant to remove [32].

SURE's approach is to fully exploit the LibOS-based design of unikernels by deploying the sidecar as a library linked into the function code within the unikernel. The unikernel's single-address-space design eliminates boundary crossings between kernel and userspace and

simplifies data exchange between the library-based sidecar and user code through internal function calls [32]. The sidecar is structured around predefined hook points on the receive (RX) and transmit (TX) paths in SURE’s network I/O libraries, where cloud providers can compose sequences of sidecar functions such as request logging, metrics collection, rate limiting, and traffic filtering.

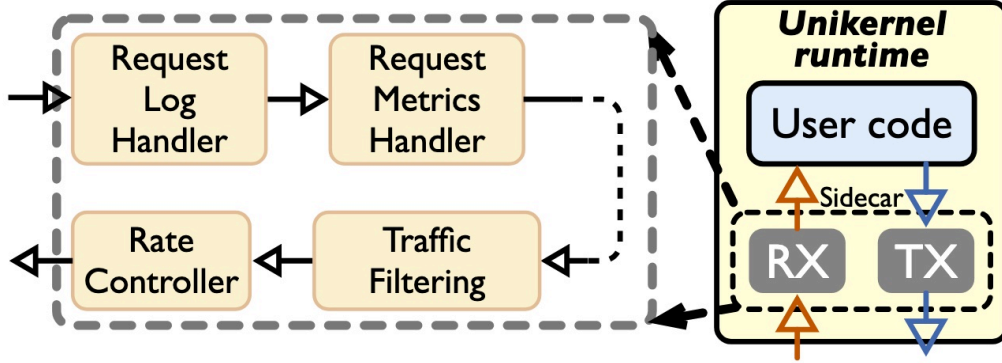


Figure 4.3: Library-based sidecar in SURE, with RX and TX hook points.

The performance argument for the library-based sidecar is striking. SURE reports that the library-based sidecar adds only 0.21–0.23 μ s of delay per request and consumes around 500 CPU cycles, while an NGINX-based sidecar adds more than 24 μ s of delay and over 60,000 CPU cycles. The throughput cost of the library-based sidecar is correspondingly negligible compared with the throughput cost of the NGINX sidecar, and the memory footprint shrinks from 20.2MB (NGINX) to 125KB (library-based), a roughly 165 $\times$ reduction [32]. Compared with an individual userspace sidecar, the paper reports more than 100 $\times$ CPU cycle savings and roughly 16 $\times$ throughput improvement on representative microbenchmarks.

4.5 MPK with a Hardened Unikernel TCB

The risks introduced by sharing an address space between user code, TCB modules, and shared memory are nontrivial. SURE identifies two threat sources from this sharing: vulnerabilities arising from shared memory processing (where a malicious function might exploit shared memory to gain unauthorized access or perform memory-based attacks) and intra-unikernel vulnerabilities arising from the single address space (where buggy or malicious code might tamper with sidecar metrics or other TCB data structures) [32]. To address these, SURE uses Intel Memory Protection Keys.

MPK allows up to sixteen 4-bit keys to be defined locally within a process. The access privilege of each key is configured in a per-core 32-bit PKRU register, where each key has an Access Disable (AD) bit and a Write Disable (WD) bit [32]. The WRPKRU instruction modifies the PKRU on the current core, switching the access privileges of all pages tagged with a given

key. SURE uses two of the sixteen keys: one (UK) tags unprotected pages, and the other (PK) tags protected pages. Most TCB memory is statically tagged with PK, while shared-memory buffers are dynamically retagged from PK to UK when they are passed into user code and back to PK when ownership is returned.

The call-gate abstraction is the second half of the design. To safely execute privileged TCB functions on behalf of user code, the call gate updates PKRU to enable access to protected pages, switches to a stack in protected memory, calls the privileged function, and then reverses the PKRU update on return. All other writes to PKRU are disallowed and enforced through binary inspection of the SURE VM’s executable, following the technique of ERIM [39].

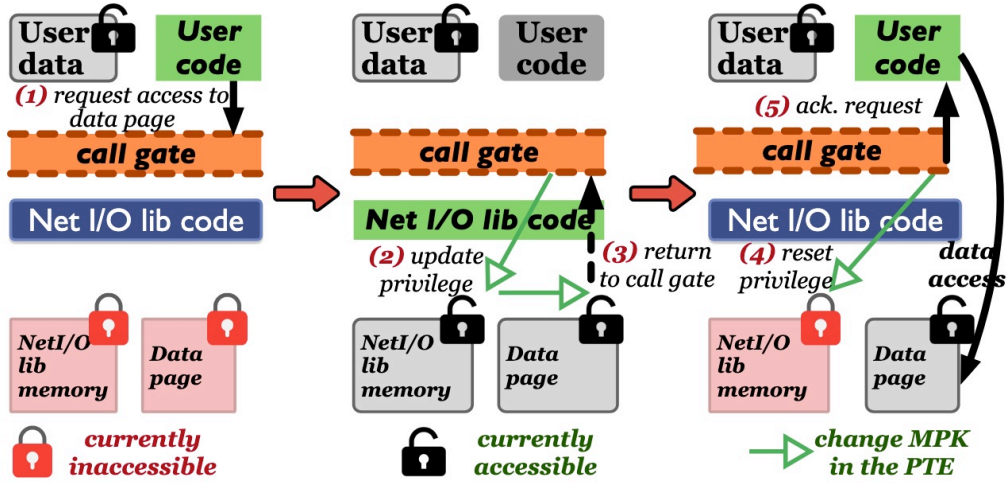


Figure 4.4: SURE’s MPK-based call gates between user code and trusted runtime modules.

MPK alone, however, is not enough. SURE’s distinctive contribution—relative to prior MPK-based unikernel work such as CubicleOS [36] and FlexOS [25]—is the observation that memory protection in a single-address-space unikernel can still be defeated if the components that manipulate the PKRU register or page tables are not themselves protected. An attacker who cannot write directly to a protected page can still try to relabel the page (so that its existing protection key no longer applies), to corrupt a stored PKRU value (so that a context switch restores attacker-chosen privileges), or to execute a stray WRPKRU instruction inside trusted code (so that the privilege boundary is crossed without going through a call gate). Each of these requires defending a different sensitive structure inside the unikernel TCB. The paper identifies four categories of TCB hardening that close these paths [32]:

1. **Paging API protection.** Untrusted code might modify the MPK key in a PTE or remap the physical address of a protected page. SURE introduces a blacklist of protected physical addresses, tags pages containing page tables with the protected key, and exposes a trusted paging API guarded by call gates. Modifications to PTEs that touch blacklisted pages cause the unikernel to crash.

2. **Scheduler protection.** Thread context switches store and load PKRU register values, so the scheduler must run as privileged code, the thread context must reside in protected memory throughout its lifecycle, and untrusted code may only access the scheduler through call gates.
3. **Interrupt validation.** Upon entering an interrupt service routine, the PKRU register on the current core retains the privileged value of the interrupted thread. To prevent an attacker-controlled ISR from gaining unallowed privileges, SURE extends the entry code common to all ISRs to set PKRU to the unprivileged value, store the (instruction pointer, stack pointer) pair in protected memory, and validate it on exit.
4. **Binary inspection of MPK-related instructions.** A strict write-xor-execute memory policy combined with binary inspection ensures that WRPKRU and XRSTOR appear only at sanctioned sites and that untrusted code cannot manipulate PKRU outside the call gate.

This enhanced TCB is what gives SURE the right to claim that its MPK-based protection is robust rather than merely advisory. It is also one of the most technically distinctive parts of the paper, because it identifies a concrete vulnerability in prior MPK-based unikernels and proposes a concrete remedy.

4.6 Microbenchmarks and the Online Boutique

SURE’s evaluation is centered on data-plane efficiency, sidecar overhead, the cost of MPK protection, and end-to-end performance on a representative microservice workload. The measurements are taken on NSF Cloudlab nodes with 32-core Intel Xeon Silver 4314 CPUs (which support MPK), 128GB of memory, and a 100Gb NIC, running Ubuntu 22.04 with kernel 5.15; the comparators span containerized and OSv-based serverful baselines and three serverless platforms (Knative, SPRIGHT, NightCore) covering the kernel-networking, eBPF-accelerated, and engine-mediated approaches [32]. The paper reports four main results.

First, on a microbenchmark of intra-node performance, SURE achieves the lowest latency and highest throughput among the alternatives. Round-trip latency is around 14–16 μ s and is essentially flat with respect to message size up to 8KB; alternatives based on Linux bridges or userspace OVS show higher latencies that grow with message size [32].

Second, MPK in SURE has limited overhead. With a single connection, MPK adds 1.2–1.3 $\times$ delay; with 64 concurrent connections, the throughput drops by roughly 1.8 $\times$ [32]. The paper notes that switching access privileges via WRPKRU costs only about 20 CPU cycles, while a comparable mprotect-based approach would cost more than 1000 CPU cycles. This is a useful number to remember when reasoning about whether a security mechanism can be placed on the data plane at all.

Third, the library-based sidecar is dramatically cheaper than an individual NGINX sidecar, both in CPU cycles and in added latency, as discussed in Section 4.4.

Fourth, on the realistic Online Boutique microservice workload, SURE achieves up to 79 $\times$ throughput improvement over Knative [21], up to 17 $\times$ improvement over a containerized serverful baseline (gRPC over kernel networking), and roughly 6 $\times$ improvement over SPRIGHT [35] in inter-node settings where SPRIGHT must fall back to the kernel TCP/IP stack [32]. Tail latency for SURE remains sub-millisecond even at concurrency levels that saturate alternatives. The choice of comparators reflects SURE’s positioning as the next step beyond cold-start-focused work such as Catalyzer [11], SEUSS [7], and SAND [2], which target startup latency rather than steady-state communication.

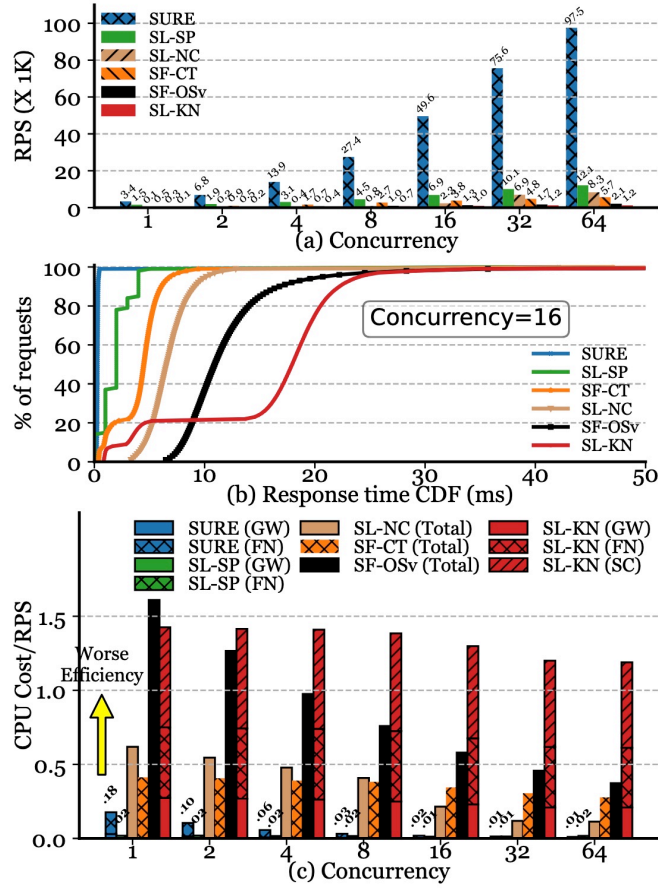


Figure 4.5: Online Boutique evaluation, intra-node setting: (a) RPS, (b) response-time CDF, (c) CPU cost per RPS.

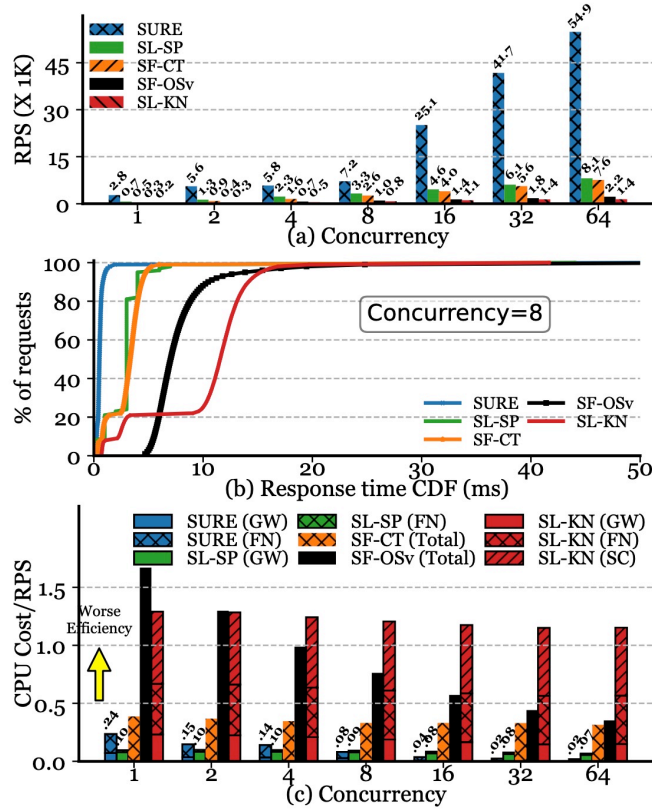


Figure 4.6: Online Boutique evaluation, inter-node setting (panels as in Figure 4.5).

The startup analysis is also informative. Boot time of a SURE VM is dominated by the VMM and BIOS, so the guest-only boot time is roughly comparable to OSv on Firecracker [32]. The paper notes that SURE’s shared-memory data plane currently cannot run on Firecracker because Firecracker does not support inter-VM shared memory devices, which means that the design as described is not directly deployable in Lambda. This is an interesting boundary condition to keep in mind when comparing the two papers.

4.7 What SURE Buys and What It Demands

The strengths of SURE are unusually intertwined with its diagnosis of the problem. The paper’s most valuable move is its insistence that serverless overhead has to be measured along the whole data path, not only at the sandbox boundary. Production traces of thousands of inter-dependent microservices in a single workload [32] make that critique credible rather than rhetorical, and the resulting design is a genuine systems-level integration of runtime, protocol stack, and service mesh rather than a stack of point optimizations. Just as importantly, security in SURE lives on the fast path: MPK enforces per-page memory ownership and isolates trusted runtime modules from untrusted user code, and the TCB-hardening analysis turns “we use MPK” into a contribution in its own right by closing privilege-escalation paths that prior MPK-based unikernels had

largely overlooked.

These benefits, however, come at the cost of an unusually specialized execution environment. SURE assumes a unikernel-based runtime model and asks the developer to build the function against its libraries rather than as an unmodified Linux binary. It depends on hardware MPK, which is not available on every CPU. And it introduces architectural complexity in the coordination of shared memory, gateway processing, library-based sidecars, and protected runtime components, each of which must be implemented and maintained as part of the platform. The paper is also explicit about a deployment-level constraint: today, SURE’s shared-memory data plane requires inter-VM shared memory device support, which is not available in Firecracker, so the strongest performance claims depend on a non-Firecracker VMM even though Firecracker is the de facto baseline for production serverless. A more fundamental caveat is workload sensitivity: SURE’s design pays off most when communication overhead dominates, when functions are colocated and frequently chained, and when L7 sidecar functionality is required. For workloads dominated by individual function execution rather than inter-function communication, the gains shrink and the cost of specialization may not be justified.

The cleanest way to position SURE is therefore as a shift from virtualization minimization to end-to-end serverless specialization. Where Firecracker narrows the VMM and accepts a conventional Linux guest, SURE rethinks the serverless runtime as a secure communication substrate, with intra-runtime protection that makes single-address-space unikernels safe for multi-tenant deployment. This is a meaningful evolution, but it is also a different bet about which part of the stack is worth specializing: Firecracker bets on the VMM, SURE bets on the data plane and the runtime internals, and—as we will see—LITESHIELD bets on the user-to-host interface itself.

Chapter 5

LITESHIELD

5.1 Re-asking Where the Boundary Should Be

LITESHIELD begins from a different question than the previous two papers. Containers remain attractive because of portability, density, and low operational cost, yet their direct dependence on the host kernel makes them difficult to use as the primary isolation mechanism in multi-tenant clouds: the default seccomp whitelist for Docker still permits more than 250 syscalls, which is large enough to be a meaningful attack surface [28]. Existing solutions respond by placing containers inside VMs (e.g., Kata Containers [31]), by minimizing the guest kernel as Firecracker does, or by adopting a userspace guest kernel as gVisor does [40], but each of these still retains a hypervisor, a full guest kernel, or a substantial userspace re-implementation of the kernel interface [28]. LITESHIELD instead asks whether secure containers can be built by reexamining the boundary between user applications and system services—an architectural question rather than a tuning question, and what makes its contribution different in kind from Firecracker’s specialization or SURE’s data-plane reorganization. The paper also draws on a wider microkernel-style tradition, including seL4 [20] and Arrakis [33], that has argued for moving system services out of the privileged kernel.

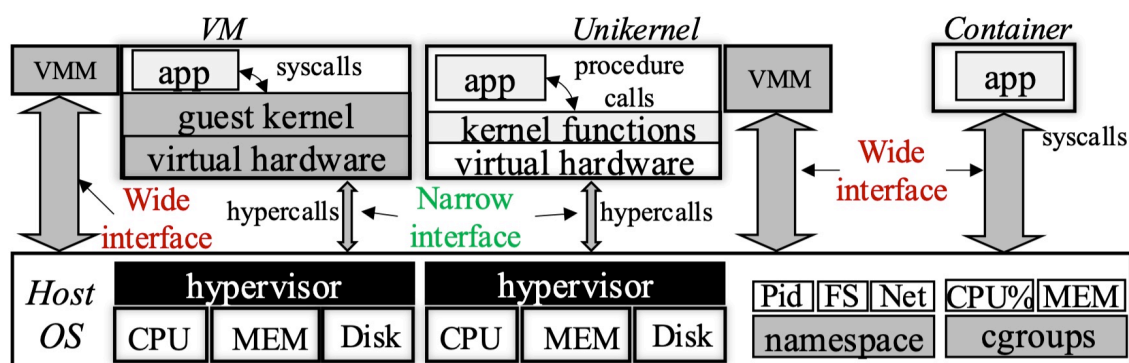


Figure 5.1: Three representative isolation architectures: VM, unikernel, and container.

The paper’s threat model is deliberately concrete. It trusts hardware-level protections such as page tables and CPU execution modes, focuses on software deficiencies in host kernels,

guest kernels, and hypervisors that can be exploited through user-to-host interfaces, and—most importantly—evaluates attack surface using two measurable metrics: the size of the user-to-host interface and the amount of code accessible through it [28]. The two metrics are deliberately operational. Counting syscalls captures the breadth of the privileged interface that an attacker can drive directly, while measuring the code reachable behind that interface captures the depth of kernel paths that a single allowed syscall can traverse. Together they give a quantitative target: a 22-syscall interface paired with a small, audited userspace service codebase makes a claim that a 250+ syscall container interface backed by millions of lines of monolithic kernel code cannot. Side-channel attacks are explicitly out of scope and treated as an orthogonal concern to be addressed by complementary mechanisms. This narrowing is what lets the paper turn “how secure is this?” into something that can be reasoned about quantitatively.

5.2 Userspace μ Kernel Services and a Thin Host Interface

LITESHIELD decouples traditional guest-kernel functionality into modular userspace microkernel (μ kernel) services that interact with guest applications through low-latency, shared-memory IPC [28]. The central claim is that by serving most Linux syscalls in userspace, the system can drastically shrink the user-to-host interface while avoiding the complexity of hypervisors and hardware virtualization. The implementation is roughly 7,000 lines of C/C++ and supports about 170 Linux kernel syscalls (142 delegable and 28 non-delegable), with most remaining unsupported syscalls requiring root privileges that ordinary guest applications do not need.

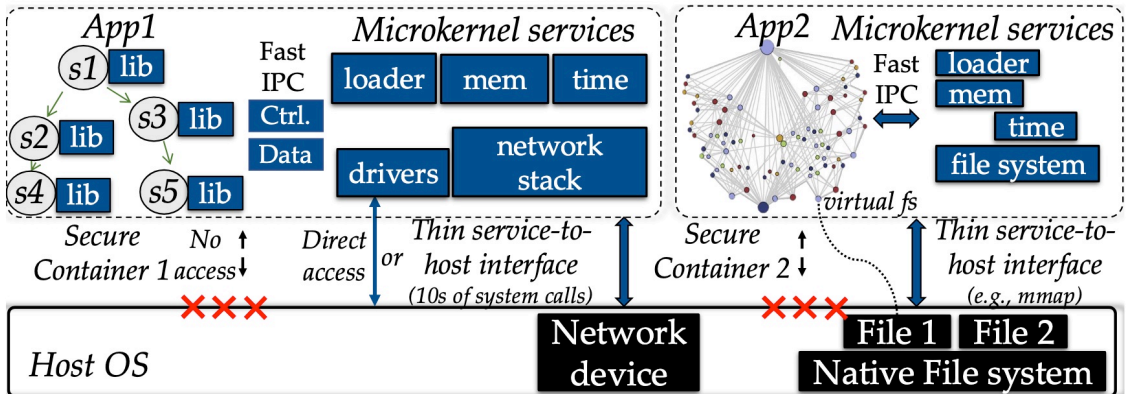


Figure 5.2: Architecture of LITESHIELD.

LITESHIELD inherits the operational advantages of the microservices architecture: modular development, flexible deployment, and rapid iteration. Userspace μ kernel services can be gradually and individually developed, extended, replaced, customized, and integrated with other userspace systems, including specialized network stacks and file systems [28]. This is a significant operational departure from monolithic guest kernels, which tend to be highly optimized

but resistant to specialization.

The isolation argument itself rests on three reinforcing layers. First, guest applications have no direct access to the host kernel: a default-deny seccomp profile is applied to each application, blocking all direct syscalls. Second, userspace μ kernel services are permitted to access the host kernel only when necessary, and they too are constrained by a restrictive seccomp profile that allows only a minimal set of explicitly defined syscalls [28]. Third, even if a malicious guest application exploits a μ kernel service through an IPC channel, it can only gain access to a restricted userspace process, providing defense in depth analogous to the guest-kernel-plus-hypervisor structure of VMs. The aggregate effect is a host-facing interface of only twenty-two syscalls, a dramatic contraction from the hundreds typically exposed by containers and comparable in spirit to the narrower privileged interfaces of VM-based systems (KVM-based VMs use roughly 20 hypercalls and 60+ VMExits). The paper’s appendix provides a more detailed breakdown of which categories of syscalls can be delegated to userspace and which cannot.

5.3 Fast IPC and Composable Specialization

The thin-interface story would not work without an IPC mechanism that makes userspace service calls cheaper than syscalls. Each application is assigned a shared memory buffer, and when the application makes a request, it places the syscall number and arguments in the shared buffer and toggles a flag that lets a μ kernel polling thread know a request is pending. The μ kernel service handles the request, places the response in the buffer, and toggles the flag again. The mechanism is inspired by techniques such as RDMA and uses dedicated polling threads on separate cores to minimize latency [28].

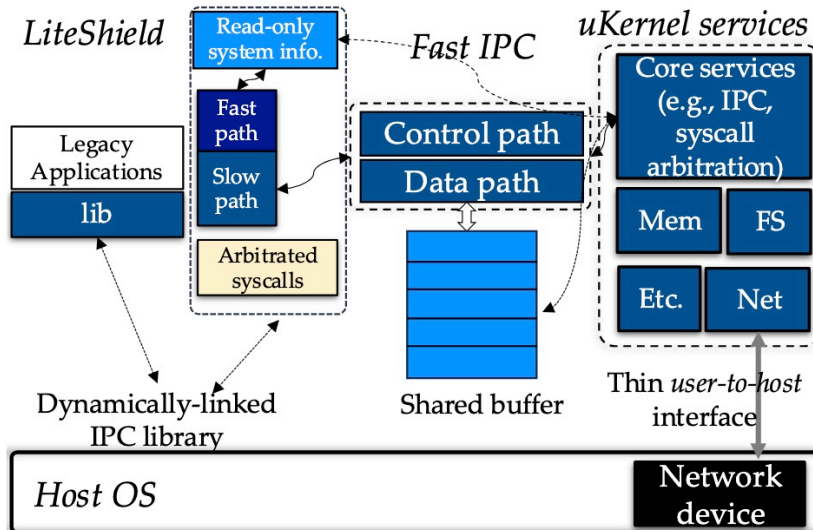


Figure 5.3: Fast userspace IPC mechanism in LITESHIELD.

The performance argument depends on hardware caches. A traditional syscall on x86 must save and restore the user-mode thread state, switch CR3 if Kernel Page-Table Isolation is enabled, traverse a kernel entry path, and eventually return through symmetric machinery—several hundred to a few thousand cycles even on the optimized fast path, with cache pollution on top. LITESHIELD’s IPC instead sets a request word in a shared cache line and waits for a polling thread on a sibling core to observe the change. When the producer and consumer cores share a last-level cache, the transfer becomes a cache-to-cache forwarding of a single line—tens of cycles, with no privilege transition and no TLB flush [28]. The catch is that polling consumes a full core whether or not requests arrive, so the design implicitly assumes a deployment in which dedicating one or two cores per worker to system-service polling is acceptable. In the densely packed serverless setting, this assumption matches reality; in latency-insensitive batch settings it would not.

A useful contrast is with FlexSC [37], which also rethinks the syscall interface but stays inside a single privileged kernel: applications post syscalls into a shared ring and a kernel thread services them asynchronously, removing the per-call exception while keeping the kernel as the executor. LITESHIELD takes the opposite route. It moves the executor into userspace μ kernel services and uses fast IPC plus selective syscall trapping (via ptrace) to intercept and redirect calls, while remaining compatible with unmodified Linux and legacy applications [28]. The contrast is illuminating: FlexSC narrows the cost of crossing the boundary, while LITESHIELD removes most of those crossings altogether and reasons about what should remain. The interception path itself draws on a body of work on safe syscall interposition, including sysfilter [10], that emphasizes the difficulty of doing this without compromise.

The μ kernel services themselves split into two categories. Core services—IPC, syscall arbitration, time and memory management—are required by every guest application. Composable services—file, device, and networking management—are provided on demand. The paper demonstrates the feasibility of composability through two concrete integrations: a port of the DPDK-based userspace network stack f-stack [12], which required only around 400 additional lines of code to integrate into the IPC machinery, and an ext2-like userspace filesystem developed from scratch. Both sit behind LITESHIELD’s IPC interface, so applications interact with them through the same shared-memory mechanism as with core services. This composability is a striking architectural property: specialized userspace approaches in the broader OS literature—Snap-style host networking [30] and persistent-memory file systems such as SplitFS [17]—can be plugged into the LITESHIELD runtime without redesigning the rest of the system. Where Firecracker accepts a general-purpose guest kernel and SURE accepts a unikernel-based runtime highly specialized for serverless, LITESHIELD makes specialization a first-class architectural property of the runtime itself.

5.4 Engineering for Compatibility

The need to handle non-delegable syscalls and legacy applications is the most subtle part of the LITESHIELD design. Some Linux syscalls cannot easily be delegated to a separate process under an unmodified Linux kernel. Process management calls (`fork`, `clone`, `wait`, `exit`) and memory management calls (`mmap`, `munmap`, `mprotect`, `msync`, `madvise`) typically need to execute in the context of the calling process [28]. LITESHIELD’s response is twofold.

First, the system distinguishes delegable syscalls (which are redirected to userspace μ kernel services) from non-delegable syscalls (which must be executed in the calling process). The split tracks Linux’s process-as-resource-namespace assumption rather than a property of the syscalls themselves: process management calls (`fork`, `clone`, `wait`, `exit`) and address-space calls (`mmap`, `munmap`, `mprotect`, `msync`, `madvise`) act on the calling process’s own state, so executing them inside a separate μ kernel service would simply produce the wrong effect. Of the roughly 170 supported calls, 142 are delegable and 28 are non-delegable. For non-delegable syscalls, LITESHIELD permits the call in the seccomp profile but uses the Linux `ptrace` mechanism to trap, monitor, and validate it through the core μ kernel service before the syscall proceeds. This adds 15–35 μ s of overhead per non-delegable call, with the upper end of the range coming from cheap calls such as `mmap` and `futex` where `ptrace` overhead dominates the syscall’s own cost, and the lower relative end from heavier calls such as `fork` and `clock_nanosleep` where the underlying work amortizes the `ptrace` cost [28]. Because non-delegable calls are infrequent on the hot path of typical applications, the absolute overhead is acceptable, and the paper notes that a kernel-module-based scheme could later convert non-delegable calls into delegable ones and remove the cost altogether.

Second, to support legacy applications without binary modification, LITESHIELD provides a POSIX-compatible library that combines `LD_PRELOAD` with the binary translation capabilities of `libsycall_intercept`. `LD_PRELOAD` injects the library into the address space of legacy applications at runtime, while `libsycall_intercept` provides fine-grained syscall interception by hooking directly into the syscall execution path through inline rewriting [28]. The combined mechanism enables LITESHIELD to dynamically intercept delegable syscalls in unmodified binaries and redirect them to userspace μ kernel services through the IPC shared buffer. The mechanism does have edge cases: statically linked binaries do not load `LD_PRELOAD` libraries; binaries with custom `libc` implementations may bypass the wrappers; and inline-assembly syscall instructions are not visible to the dynamic linker. Such applications run into seccomp’s default-deny profile and fail, and the paper outlines a hotpatching technique—disassembling syscall instruction sites in such binaries and rewriting them with hooks—that would close the gap, though as explicit future work.

This compatibility story is especially important when LITESHIELD is compared with

SURE, which requires applications to be relinked against unikernel libraries, and even with Firecracker, which preserves Linux compatibility but does so by running a full guest kernel inside a VM.

5.5 What the Numbers Show

LITESHIELD’s evaluation is structured around interface reduction, syscall latency, network and filesystem performance, and a representative real application. The paper compares LITESHIELD with Docker containers, KVM-based VMs, Firecracker, and gVisor on a server with an Intel Xeon Gold 6430 CPU, 96GB DDR5 RAM, and an NVMe SSD running Ubuntu 22.04 with kernel 5.15. KVM tests use 16 vCPUs and 32GB of RAM, gVisor v1.10.1 is tested in systrap mode, Firecracker v1.10.1 is tested with an Ubuntu 24.04 root filesystem on Linux kernel 5.10, and Docker and LITESHIELD are constrained to 16 cores and 32GB of RAM via cgroups [28].

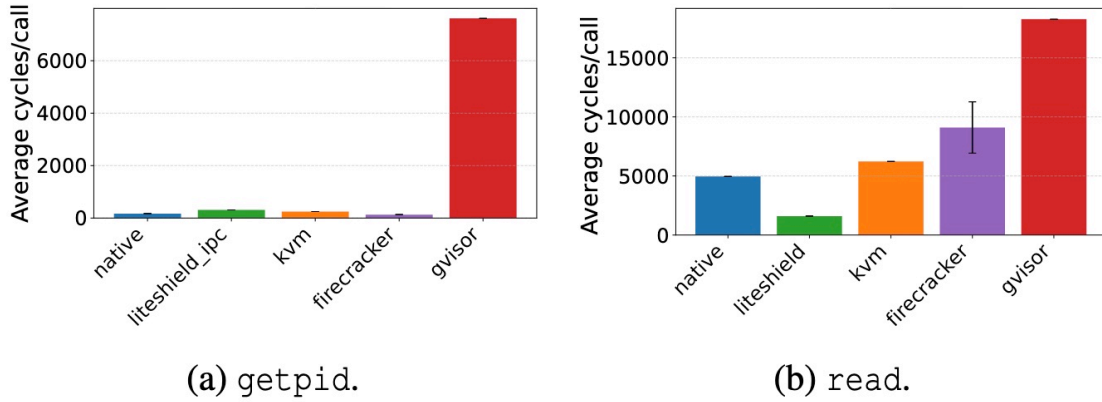


Figure 5.4: Syscall latency: `getpid` (simple) vs. `read` (complex).

The paper reports four main findings. First, on simple syscalls such as `getpid`, LITESHIELD achieves significantly lower latency than gVisor (a userspace guest-kernel design) while remaining comparable to VM-based approaches. On a complex syscall such as reading one byte from each block of a 4GB file, LITESHIELD outperforms VM-based approaches because reads no longer trigger VMExits, and it can even surpass native performance because of its specialized lightweight userspace filesystem [28].

Second, `ptrace`-based arbitration of non-delegable syscalls introduces 15–35 μ s overhead per call, more pronounced for lightweight syscalls (`mmap`, `futex`) and less significant for heavier ones (`fork`, `clock_nanosleep`) [28].

Third, the userspace network stack ported from `f-stack` provides UDP performance that outperforms all other isolation mechanisms and is only slightly below native, while TCP performance with `iperf` is comparable for small packets but lags microVM approaches for larger

packets because f-stack lacks Generic Receive Offload (GRO). The paper notes that this result reinforces the absence of a one-size-fits-all solution: LITESHIELD allows applications to select μ kernel services tailored to their needs [28].

Fourth, on Redis 6.0.16 under YCSB across four workloads (mixes of reads, updates, inserts, and read-modify-write), LITESHIELD reports higher performance than native execution *in this specific configuration*. The result looks counterintuitive but follows directly from the architecture: Redis on these workloads is dominated by short, frequent syscalls (`epoll_wait`, `read`, `write`), and on a native Linux container each of these pays the per-call kernel-entry tax. Under LITESHIELD, the same operations resolve through cache-line IPC to a userspace network and filesystem service, so the marginal cost per operation drops below what a native syscall would have paid. The same reasoning explains why Firecracker and gVisor sit further behind: their I/O paths add VMExits or userspace-to-userspace round trips on top of the native syscall cost rather than replacing it. This finding should not be generalized to “LITESHIELD is faster than native”: it depends on the specific Redis/YCSB workload mix and on the userspace network and filesystem services LITESHIELD ships with. For workloads that are CPU-bound, that allocate large memory buffers, or that already avoid frequent syscalls, the gap with native is expected to close or invert [28].

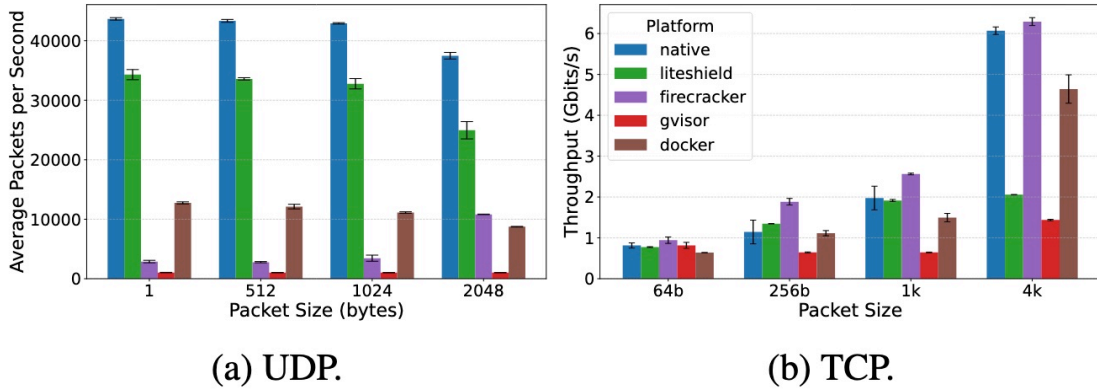


Figure 5.5: Network performance on UDP and TCP across packet sizes.

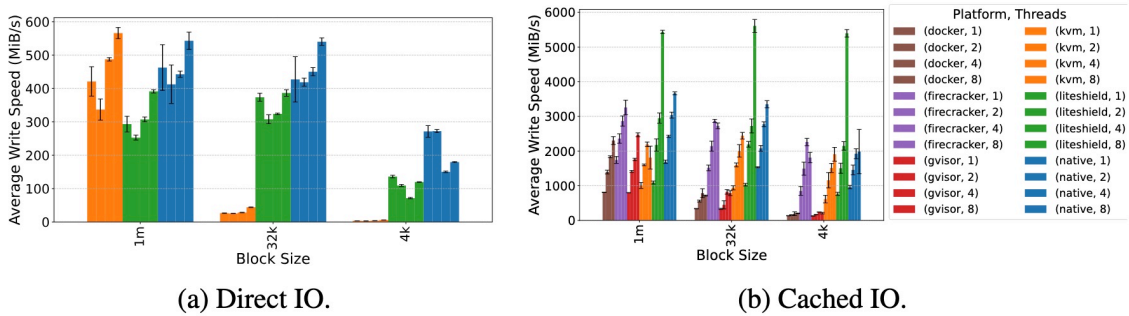


Figure 5.6: Filesystem write performance under direct I/O and cached I/O.

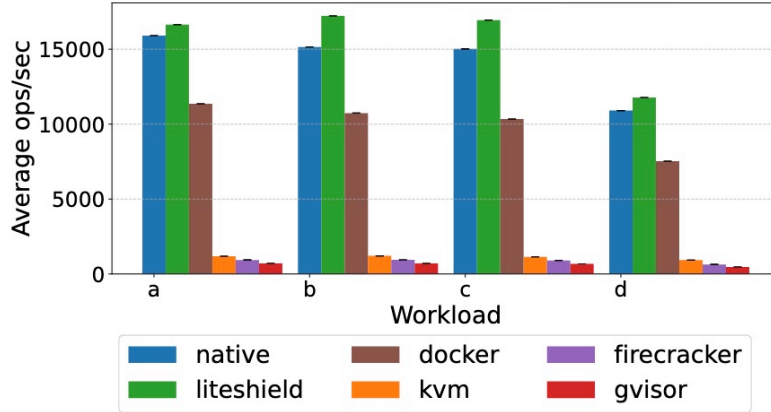


Figure 5.7: YCSB performance on Redis under LITESHIELD.

The evaluation logic here differs from both Firecracker and SURE. LITESHIELD is less concerned with microVM boot rates or serverless gateway throughput than with proving that strong isolation can be approximated through a thin userspace-service architecture that remains compatible with existing kernels and legacy applications. The set of comparators (Docker, KVM, Firecracker, gVisor) is also chosen to cover the four major points of the existing isolation design space, which makes the comparison more credible than a comparison against any single baseline would be.

5.6 LITESHIELD as a Reframing, Not a Refinement

The most distinctive thing about LITESHIELD is that it redirects attention to a more measurable security-relevant quantity. Where prior work treats “does the system use a hypervisor?” as the security question, LITESHIELD treats the size of the user-to-host interface as the attack surface that can actually be measured, contracted, and reasoned about. The 22-syscall result is more than a benchmark figure; it operationalizes a conceptual move from binary security categorization to quantitative interface analysis. Three further design properties follow from this move and reinforce each other: a modular μ kernel-service architecture in which specialization is first-class (the 400-line f-stack port being the tangible evidence); a serious compatibility engineering effort that addresses the very objection—unmodified Linux binaries—that has historically pushed isolation work back toward general-purpose VMs; and an honest performance analysis that measures ptrace overhead directly rather than reporting only favorable workloads, while pointing toward kernel-module-based mechanisms that could eliminate the remaining overhead in future.

The same architectural choices, however, define the boundary of the argument. Avoiding hypervisor complexity comes at the cost of introducing a new userspace-service decomposition layer that must be secured, maintained, debugged, and integrated with application behavior;

shared-memory IPC, polling threads, and interception logic are efficient but not free of operational complexity, and the cost of operating them at production scale is not yet established by the paper. Performance parity with containers does not automatically imply superiority under every workload—LITESHIELD pays off when the reduced interface and modular service structure outweigh the engineering cost of the new runtime model, and that calculation depends on the deployment. The paper itself flags that statically linked applications, applications with custom libc implementations, or those using inline-assembly syscalls can bypass the interception library and fail under seccomp; a hotpatching technique that rewrites syscall instructions in such binaries is presented as future work. A more conceptual limitation is that the threat model excludes side and covert channels and assumes an unmodified Linux kernel; comparing LITESHIELD with hypervisor-based architectures under a unified threat model that includes microarchitectural attacks remains genuinely open.

The clearest way to read LITESHIELD is therefore as a reframing rather than a refinement. Firecracker minimizes the VMM, SURE specializes the serverless runtime and data plane, and LITESHIELD decomposes the kernel-service layer itself into userspace components while keeping the host interface thin and removing the hypervisor entirely. It is a distinct and conceptually ambitious point in the cloud-native isolation design space, and the design is more clearly an alternative to hypervisor-based isolation than a refinement of it. By serving most system functionality through composable userspace services, by sharply narrowing the host interface to roughly twenty-two syscalls, and by integrating shared-memory IPC, ptrace-based arbitration, and runtime injection, LITESHIELD offers an interpretation of “lightweight yet strong” isolation that is genuinely different in kind from both Firecracker’s VMM minimization and SURE’s unikernel-centered runtime.

Chapter 6

Cross-Paper Synthesis and Open Challenges

6.1 Three Axes of Variation

The three focal papers can be compared along three broad axes: boundary location, interface design, and specialization target.

On the first axis, Firecracker places the main protection boundary at the microVM level, supported by KVM. SURE places a VM boundary around each function but also strengthens protection inside the unikernel runtime through MPK-based call gates and a hardened TCB. LITESHIELD removes the hypervisor entirely and uses process isolation, seccomp, and userspace-service decomposition to move much system functionality out of the traditional kernel path while keeping a thin interface to the host. The boundary is no longer a single fact about a system; it is a layered structure in which each paper adds, rearranges, or removes different layers.

On the second axis, the papers differ in what they optimize on the hot path. Firecracker narrows the virtualization stack and device model but still leaves a full guest kernel between application and hardware. SURE targets communication-heavy serverless paths through zero-copy shared memory, userspace protocol processing, library-based sidecars, and MPK-protected runtime modules. LITESHIELD targets the breadth of the user-to-host interface and replaces repeated syscall crossings with shared-memory IPC. Each paper is making a different bet about which crossings are most worth avoiding.

On the third axis, specialization becomes progressively more explicit. Firecracker is specialized for serverless and containers but preserves a conventional Linux guest model. SURE is specialized for function chaining and serverless data-plane efficiency, with most of its design making sense only when communication overhead dominates. LITESHIELD is specialized around modular userspace services and a thin-interface secure-container model, with the explicit ambition of letting future workload-specific services be plugged in without redesigning the system.

Table 6.1 summarizes these three axes under the common analytical framework of Chapter 2.

Table 6.1: Comparison of the three focal papers under the analytical framework of Chapter 2.

Paper	Primary bound-ary	Main optimiza-tion target	Compatibility posture	Signature contri-bution
Firecracker	MicroVM via KVM-backed minimal VMM	Reduce virtual-ization overhead, startup time, and per-instance footprint	Strong support for unmodified Linux guest and kernel userland	Minimal mi-croVM monitor specialized for serverless and containers
SURE	Unikernel-based VM plus MPK-protected intra-runtime components and TCB	Accelerate secure function chaining and service-mesh communication	More spe-cialized unikernel run-time model; applications relinked	Distributed zero-copy data plane, library-based sidecar, MPK-protected runtime with hardened TCB
LITESHIELD	Restricted userspace pro-cesses with seccomp and thin host interface; no hypervisor	Reduce user-to-host interface and replace syscall crossings with shared-memory IPC	Legacy ap-plication support via interception (LD_PRELOAD + lib-syscall_intercept) and ptrace ar-bitration	Composable userspace micro-services for secure con-tainers with ≈ 22 host syscalls

This table makes explicit a point that is otherwise easy to lose. The three papers are not three implementations of the same idea. They are three different architectural answers to a shared family of questions, and each answer commits to a different combination of compatibility, hardware substrate, performance focus, and trusted-computing-base structure.

6.2 Threat Models in Comparative Form

Because each paper makes its security argument on a different layer of the stack, their threat models are easy to talk past. Table 6.2 aligns them by viewing each system through the cloud-provider’s lens—what is trusted, what is treated as adversarial, and what is explicitly out of scope. The contrast clarifies that “stronger isolation” has different operational meanings across the three designs.

Table 6.2: Threat models of the three focal papers, viewed from the cloud provider’s perspective.

System	Trusted	Untrusted	Out of scope
Firecracker	Host kernel/KVM and the Firecracker VMM (Rust, $\approx 50K$ LoC)	Guest kernel and tenant workload inside each microVM	Microarchitectural side and covert channels; non-malicious operator error
SURE	Hypervisor and the hardened unikernel TCB (paging API, scheduler, ISR, MPK code)	User function code inside each SURE VM, including its access to shared memory	Hardware-level MPK escapes; microarchitectural attacks on PKRU; side channels
LITESHIELD	Unmodified host kernel plus a small set of μ kernel services and the IPC/arbitration layer	Guest application processes (including legacy binaries served by interception)	Side and covert channels; attacks against an unmodified host kernel itself

The application-side TCB story differs from the provider-side story for each system. In Firecracker, the application additionally trusts its own guest kernel; in SURE, it additionally trusts the unikernel libraries it is linked against; in LITESHIELD, it trusts the μ kernel services it talks to over IPC. The provider-side framing in Table 6.2 is the more useful one for cross-paper comparison, because it isolates what each system is actually defending in a multi-tenant deployment.

6.3 Recurring Themes Across the Three Papers

The three papers are also linked by direct technical relationships, not just by thematic resemblance. LITESHIELD is benchmarked against Firecracker and gVisor in addition to Docker and KVM-based VMs, and explicitly cites Firecracker as the canonical example of microVM-based minimization that nonetheless retains a guest kernel [28]. SURE cites Firecracker as a microVM VMM and notes that Firecracker’s lack of inter-VM shared memory device support is a non-trivial obstacle to deploying SURE’s data plane in Lambda-class environments [32]. Conversely, both LITESHIELD and SURE inherit elements of Firecracker’s stance—they accept that production isolation must be both lightweight and strong, that minimal trusted code is preferable to maximal flexibility, and that the workload to optimize for is cloud-native rather than general-purpose.

Several technical themes recur across the three papers and become more visible when read together:

- **Reducing the trusted computing base.** Firecracker reduces the VMM to roughly 50K lines of Rust. SURE reduces the runtime to a unikernel built on Unikraft and explicitly hardens the unikernel's TCB. LITESHIELD reduces the host-facing TCB to a small set of μ kernel services and a 22-syscall interface. Each paper is, in a different way, trying to make the security argument depend on as little code as possible.
- **Replacing kernel crossings with cheaper alternatives.** SURE replaces socket-based sidecar communication with internal function calls inside a unikernel and replaces kernel TCP/IP with Z-stack. LITESHIELD replaces syscalls on the hot path with shared-memory IPC backed by polling threads. Firecracker does not directly do this inside its guest, but it does narrow the host-facing syscall surface to roughly 24 syscalls and 30 ioctls in the jailer.
- **Co-designing with hardware.** Firecracker depends on KVM and standard hardware virtualization extensions. SURE depends additionally on Intel MPK and on DPDK polling for high-throughput packet handling. LITESHIELD depends on multi-core systems with effective last-level caches and on Linux ptrace for non-delegable syscall arbitration. The hardware substrate is no longer incidental; it shapes what the system can claim.
- **Explicit treatment of compatibility.** Firecracker preserves Linux guests. SURE accepts a porting cost (functions must be linked against SURE libraries) but argues that the resulting performance and security gains justify it. LITESHIELD takes a hybrid stance: it keeps the kernel unmodified and supports legacy binaries through interception, while still requiring a new userspace runtime layer.

The cumulative impression from these recurring themes is that the cloud-native isolation problem has stopped being a single design question and has become a family of related decisions that must be made together. The three papers can be read, in this sense, as three explorations of the same design space rather than three competing claims on a single bench.

6.4 Workload Fit and Practical Trade-offs

The three papers suggest that there is no single best answer to cloud-native isolation. Firecracker is especially attractive when broad Linux compatibility, deployment maturity, and operational tooling are central. Its minimal feature set and well-understood security boundary make it a good default for general serverless and container hosting at scale, and the production track record at AWS gives it credibility that academic systems rarely have.

SURE is attractive when serverless communication overhead dominates the workload and when the platform can support a more specialized unikernel stack. Its integrated design across runtime, data plane, sidecar, and TCB hardening is most valuable in deployments where function chains are long and where small per-message savings compound. The cost is a more specialized runtime model, more demanding hardware assumptions, and a porting burden for application developers.

LITESHIELD is attractive when one wants a thinner host interface and more modular system-service specialization without paying the full conceptual cost of hypervisor-based virtualization. Its userspace microkernel services are particularly compelling for environments where legacy applications must continue to run unmodified, where future specialized userspace services (for example, persistent-memory file systems or workload-specific network stacks) may be plugged in over time, and where the engineering team can afford the maintenance burden of a new runtime layer.

A useful way to interpret these three positions is in terms of workload fit rather than dominance. Firecracker, SURE, and LITESHIELD each work best in different regions of the deployment space, and a sophisticated platform might use elements of all three.

6.5 Open Problems

Several open problems emerge from the comparison.

Table 6.3: Open problems suggested by the three focal papers.

Open problem	Why it matters
Compatibility	Specialized runtimes pay either a porting cost or an interception cost; whether either scales to the full Linux ecosystem is unproven.
Unified evaluation	MicroVMs, unikernels, and userspace-service systems use different metrics and workloads.
Security of bypass	SURE and LITESHIELD both bypass conventional boundaries via shared memory.
Side-channel threats	All three papers treat side channels as orthogonal.
Hardware dependence	Reliance on features such as MPK or specific virtualization extensions constrains which CPU SKUs and deployment targets remain viable.
Operational integration	Production success depends on orchestration, tracing, debuggability, and warm-pool dynamics—territory only Firecracker has demonstrably navigated at scale.

6.6 What These Papers Tell Us About the Field

One broader lesson is that the field is moving away from generic, one-size-fits-all isolation. The important question is increasingly not “VM or container?” but rather “which parts of the stack should remain general, and which should be specialized for cloud-native execution?” Firecracker answers by specializing the VMM, SURE by specializing the runtime internals and the data plane, and LITESHIELD by specializing the service boundary itself. A second lesson is that the meaning of “trusted computing base” is becoming more nuanced and depends on whose viewpoint one adopts. From the cloud provider’s perspective—which protects the host and other tenants from a possibly malicious workload—Firecracker’s TCB is essentially the host kernel/KVM plus the minimal Firecracker VMM, while the guest kernel is treated as part of the untrusted, tenant-controlled environment; SURE’s TCB is the hypervisor plus the carefully hardened unikernel modules (paging API, scheduler, ISR, and the binary-inspected MPK code) that user code can subvert only through call gates; and LITESHIELD’s TCB is the unmodified host kernel plus a small set of μ kernel services and the IPC/arbitration mechanism. From the application’s perspective, however, the workload itself trusts more: in Firecracker, the application trusts its own guest kernel as well; in SURE, it trusts the rest of the unikernel libraries it is linked against; in LITESHIELD, it trusts the μ kernel services it talks to over IPC. Each paper makes its TCB visible in a different place along both axes, and a comparison that ignores either viewpoint will be misleading. A third lesson is that hardware features are becoming load-bearing in security arguments rather than incidental to them. SURE’s correctness depends on MPK behaving as documented and on the absence of microarchitectural privilege escalation in a way that Firecracker’s correctness does not, and as more designs commit to features such as MPK, MPX, or PASID, the relationship between hardware evolution and isolation architecture becomes one of the central concerns of cloud-native systems research.

Read this way, the three focal papers are best understood as different responses to the same underlying challenge: how to build execution environments that are simultaneously secure, efficient, and deployable for cloud-native workloads. Their differences clarify the design space rather than fragment it, and the cross-paper open problems they collectively expose—evaluation consistency, hardware dependence, debuggability, side-channel integration, orchestration—are precisely the problems that the next generation of isolation research will have to address.

Chapter 7

Conclusion and Future Work

This report has examined three representative papers on secure and lightweight cloud-native isolation: Firecracker, SURE, and LITESHIELD. Although all three aim to improve the balance among isolation strength, overhead, and compatibility, they do so through markedly different strategies. Firecracker minimizes the virtualization stack while preserving a conventional guest-kernel model and grounds its claims in production deployment at AWS Lambda. SURE redesigns the serverless runtime around secure and efficient communication, treating the data plane and the unikernel TCB as first-class systems concerns and relying on Intel MPK for fine-grained intra-runtime protection. LITESHIELD redefines the boundary between applications and system services, using composable userspace microkernel services and a 22-syscalls host interface to narrow the privileged surface while removing the hypervisor.

The central conclusion of this report is that recent work is converging on a more workload-aware view of isolation. Strong protection is still necessary, but it is no longer sufficient. What matters equally is how much of the system lies on the hot path, how broad the privileged interface remains, how the runtime is internally structured, and how well the architecture matches the communication and lifecycle patterns of cloud-native workloads. “Lightweight isolation” has thus stopped meaning “the same thing implemented more efficiently” and has started meaning “a different way of carving up the stack so that the right things end up on the fast path.” The three papers do not offer a single best architecture; they offer three coherent points in a design space whose dimensions are now clearer than they were before, and any future system that combines elements of more than one of them will have to reconcile their assumptions about hardware, compatibility, and TCB structure.

Future work should therefore continue in three directions. First, research should improve evaluation consistency so that systems with different abstractions can still be compared meaningfully; ideally, this would involve workload suites that exercise both per-instance overhead (Firecracker’s traditional strength), communication overhead (SURE’s traditional strength), and host-interface overhead (LITESHIELD’s traditional strength). Second, future runtimes should better integrate observability, orchestration, and failure handling into specialized isolation designs, building on Firecracker’s lesson that operational compatibility is itself a design property. Third, designers should continue exploring how to combine strong security with specialization

not only at the sandbox boundary, but also in networking, storage, and cross-service coordination, while paying explicit attention to how dependencies on hardware features such as MPK constrain portability.

A more speculative direction is the possibility of *layered combinations*: a system that uses a Firecracker-class microVM as its outer boundary, runs a SURE-style unikernel runtime inside, and exposes a LITESHIELD-style thin user-to-host interface through which the unikernel speaks to a tightly controlled host. None of the three papers studied here proposes such a combination, but the comparative analysis suggests that the design vocabulary is now mature enough that such combinations can be discussed without incoherence. Such a layered design would also risk combining the operational complexity, hardware dependence, and porting burden of all three systems at once, so its value would depend on whether the added security and specialization benefits outweigh that cumulative cost. Whether the combination is practically worthwhile is therefore one of the more interesting open questions raised by reading these three works together.

Bibliography

- [1] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight Virtualization for Serverless Applications. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*, pages 419–434, Santa Clara, CA, February 2020.
- [2] Istemi Ekin Akkus, Ruichuan Chen, Ivica Rimac, Manuel Stein, Klaus Satzke, Andre Beck, Paarijaat Aditya, and Volker Hilt. SAND: Towards High-Performance Serverless Computing. In *2018 USENIX Annual Technical Conference (USENIX ATC '18)*, pages 923–935, 2018.
- [3] Anjali, Tyler Caraza-Harter, and Michael M. Swift. Blending Containers and Virtual Machines: A Study of Firecracker and gVisor. In *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE '20)*, pages 101–113, 2020.
- [4] Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, and Andrew Warfield. Xen and the Art of Virtualization. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)*, pages 164–177, 2003.
- [5] Luiz Barroso, Mike Marty, David Patterson, and Parthasarathy Ranganathan. Attack of the Killer Microseconds. *Communications of the ACM*, 60(4):48–54, March 2017.
- [6] Fabrice Bellard. QEMU, a Fast and Portable Dynamic Translator. In *USENIX Annual Technical Conference, FREENIX Track*, pages 41–46, 2005.
- [7] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Ap-pavoo. SEUSS: Skip Redundant Paths to Make Serverless Fast. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20)*, Article 32, 2020.
- [8] Qizhe Cai, Shubham Chaudhary, Midhul Vuppalapati, Jaehyun Hwang, and Rachit Agarwal. Understanding Host Network Stack Overheads. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 65–77, 2021.

- [9] Claudio Canella, Jo Van Bulck, Michael Schwarz, Moritz Lipp, Benjamin von Berg, Philipp Ortner, Frank Piessens, Dmitry Evtyushkin, and Daniel Gruss. A Systematic Evaluation of Transient Execution Attacks and Defenses. arXiv preprint arXiv:1811.05441, 2018.
- [10] Nicholas DeMarinis, Kent Williams-King, Di Jin, Rodrigo Fonseca, and Vasileios P. Kemerlis. sysfilter: Automated System Call Filtering for Commodity Software. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID '20)*, pages 459–474, 2020.
- [11] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. Catalyzer: Sub-millisecond Startup for Serverless Computing with Initialization-less Booting. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '20)*, pages 467–481, 2020.
- [12] The F-Stack Authors. F-Stack: A High-Performance Network Framework Based on DPDK. <https://www.f-stack.org/>, accessed 2026.
- [13] Scott Hendrickson, Stephen Sturdevant, Tyler Harter, Venkateshwaran Venkataramani, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Serverless Computation with OpenLambda. In *8th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '16)*, 2016.
- [14] The Istio Authors. Istio: An Open-Source Service Mesh. <https://istio.io/>, accessed 2026.
- [15] Zhipeng Jia and Emmett Witchel. Nightcore: Efficient and Scalable Serverless Computing for Latency-Sensitive, Interactive Microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*, pages 152–166, 2021.
- [16] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, Joseph E. Gonzalez, Raluca Ada Popa, Ion Stoica, and David A. Patterson. Cloud Programming Simplified: A Berkeley View on Serverless Computing. arXiv preprint arXiv:1902.03383, 2019.
- [17] Rohan Kadekodi, Se Kwon Lee, Sanidhya Kashyap, Taesoo Kim, Aasheesh Kolli, and Vijay Chidambaram. SplitFS: Reducing Software Overhead in File Systems for Persistent Memory. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP '19)*, pages 494–508, 2019.

- [18] Avi Kivity, Yaniv Kamay, Dor Laor, Uri Lublin, and Anthony Liguori. kvm: The Linux Virtual Machine Monitor. In *Proceedings of the Linux Symposium*, 2007.
- [19] Avi Kivity, Dor Laor, Glauber Costa, Pekka Enberg, Nadav Har’El, Don Marti, and Vlad Zolotarov. OSv—Optimizing the Operating System for Virtual Machines. In *2014 USENIX Annual Technical Conference (USENIX ATC ’14)*, pages 61–72, Philadelphia, PA, June 2014.
- [20] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal Verification of an OS Kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP ’09)*, pages 207–220, 2009.
- [21] The Knative Authors. Knative: Kubernetes-based Platform to Deploy and Manage Modern Serverless Workloads. <https://knative.dev/>, accessed 2026.
- [22] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. Spectre Attacks: Exploiting Speculative Execution. In *40th IEEE Symposium on Security and Privacy (S&P ’19)*, 2019.
- [23] The Kubernetes Authors. Kubernetes: Production-Grade Container Orchestration. <https://kubernetes.io/>, accessed 2026.
- [24] Simon Kuenzer, Vlad-Andrei Bădoiu, Hugo Lefeuvre, Sharan Santhanam, Alexander Jung, Gauthier Gain, Cyril Soldani, Costin Lupu, Ștefan Teodorescu, Costi Răducanu, Cristian Banu, Laurent Mathy, Răzvan Deaconescu, Costin Raiciu, and Felipe Huici. Unikraft: Fast, Specialized Unikernels the Easy Way. In *Proceedings of the Sixteenth European Conference on Computer Systems (EuroSys ’21)*, pages 376–394, 2021.
- [25] Hugo Lefeuvre, Vlad-Andrei Bădoiu, Alexander Jung, Stefan Lucian Teodorescu, Sebastian Rauch, Felipe Huici, Costin Raiciu, and Pierre Olivier. FlexOS: Towards Flexible OS Isolation. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS ’22)*, pages 467–482, 2022.
- [26] Yiwen Li, Brendan Dolan-Gavitt, Sam Weber, and Justin Cappos. Lock-in-Pop: Securing Privileged Operating System Kernels by Keeping on the Beaten Path. In *2017 USENIX Annual Technical Conference (USENIX ATC ’17)*, pages 1–13, Santa Clara, CA, July 2017.
- [27] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike

- Hamburg. Meltdown: Reading Kernel Memory from User Space. In *27th USENIX Security Symposium (USENIX Security '18)*, 2018.
- [28] Kaesi Manakkal, Nathan Daughety, Marcus Pendleton, and Hui Lu. LITESHIELD: Secure Containers via Lightweight, Composable Userspace μ Kernel Services. In *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC '25)*, pages 973–985, Boston, MA, July 2025.
 - [29] Filipe Manco, Costin Lupu, Florian Schmidt, Jose Mendes, Simon Kuenzer, Sumit Sati, Kenichi Yasukata, Costin Raiciu, and Felipe Huici. My VM is Lighter (and Safer) than Your Container. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)*, pages 218–233, 2017.
 - [30] Michael Marty, Marc de Kruijf, Jacob Adriaens, Christopher Alfeld, Sean Bauer, Carlo Contavalli, Michael Dalton, Nandita Dukkupati, William C. Evans, Steve Gribble, Nicholas Kidd, Roman Kononov, Gautam Kumar, Carl Mauer, Emily Musick, Lena Olson, Erik Rubow, Michael Ryan, Kevin Springborn, Paul Turner, Valas Valancius, Xi Wang, and Amin Vahdat. Snap: A Microkernel Approach to Host Networking. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP '19)*, pages 399–413, 2019.
 - [31] The OpenStack Foundation. Kata Containers: The Speed of Containers, the Security of VMs. <https://katacontainers.io/>, accessed 2026.
 - [32] Federico Parola, Shixiong Qi, Anvaya B. Narappa, K. K. Ramakrishnan, and Fulvio Risso. SURE: Secure Unikernels Make Serverless Computing Rapid and Efficient. In *Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC '24)*, pages 668–688, Redmond, WA, November 2024.
 - [33] Simon Peter, Jialin Li, Irene Zhang, Dan R. K. Ports, Doug Woos, Arvind Krishnamurthy, Thomas Anderson, and Timothy Roscoe. Arrakis: The Operating System Is the Control Plane. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI '14)*, pages 1–16, Broomfield, CO, October 2014.
 - [34] Donald E. Porter, Silas Boyd-Wickizer, Jon Howell, Reuben Olinsky, and Galen C. Hunt. Rethinking the Library OS from the Top Down. In *Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '11)*, pages 291–304, 2011.
 - [35] Shixiong Qi, Leslie Monis, Ziteng Zeng, Ian-Chin Wang, and K. K. Ramakrishnan. SPRIGHT: Extracting the Server from Serverless Computing! High-performance eBPF-based Event-driven, Shared-Memory Processing. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 780–794, 2022.

- [36] Vasily A. Sartakov, Lluís Vilanova, and Peter Pietzuch. CubicleOS: A Library OS with Software Componentisation for Practical Isolation. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*, pages 546–558, 2021.
- [37] Livio Soares and Michael Stumm. FlexSC: Flexible System Call Scheduling with Exception-Less System Calls. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI '10)*, pages 33–46, 2010.
- [38] Chia-Che Tsai, Bhushan Jain, Nafees Ahmed Abdul, and Donald E. Porter. A Study of Modern Linux API Usage and Compatibility: What to Support When You’re Supporting. In *Proceedings of the Eleventh European Conference on Computer Systems (EuroSys '16)*, 2016.
- [39] Anjo Vahldiek-Oberwagner, Eslam Elnikety, Nuno O. Duarte, Michael Sammler, Peter Druschel, and Deepak Garg. ERIM: Secure, Efficient In-Process Isolation with Protection Keys (MPK). In *28th USENIX Security Symposium (USENIX Security '19)*, pages 1221–1238, 2019.
- [40] Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. The True Cost of Containing: A gVisor Case Study. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '19)*, Renton, WA, July 2019.